IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 10 April 2026, accepted 4 May 2026, date of publication 7 May 2026, date of current version 12 May 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3691262

SURVEY

A Comprehensive Survey on Concept-Drift-Resilient Network Intrusion Detection Systems

MATEUS KOMARCHESQUI[1], VITOR GABRIEL DA SILVA RUFFO[2], DANIEL MATHEUS BRANDÃO LENT[2], VINÍCIUS FERREIRA SCHIAVON[1], MATHEUS RIKI NAKANISHI[1], GUSTAVO HIDEYUKI KITAMURA NISHIKAWA[1], LUIZ FERNANDO CARVALHO[3], AND MARIO LEMES PROENÇA JR.[1]

[1]Computer Science Department, State University of Londrina, Londrina 86057-970, Brazil
[2]Electrical Engineering Department, State University of Londrina, Londrina 86057-970, Brazil
[3]Computer Engineering Department, Federal University of Technology—Paraná, Apucarana 86812-460, Brazil

Corresponding author: Mario Lemes Proença Jr. (proenca@uel.br)

This work was supported in part by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior–Brazil (CAPES) under Grant 001, in part by the Araucaria Foundation, and in part by the National Council for Scientific and Technological Development (CNPq) of Brazil under Grant 306397/2022-6.

ABSTRACT The evolution of computer networks for service provision, through new devices and architectures, renders past traffic patterns obsolete. The shift in the statistical properties of data is known as concept drift (CD) and is bound to affect automated pattern recognition-based systems. Network intrusion detection systems (NIDS) are widely studied security solutions that model and monitor network traffic behavior, alerting when malicious activities are detected. The lack of adaptability to evolving networks renders NIDS ineffective over time, leading to operational instability and increased vulnerability. In this survey, we aim to present an empirical literature review on state-of-the-art CD-resilient NIDS. The essential steps involved in developing such systems are comprehensively examined: benchmark datasets, NIDS development, drift detection and adaptation, and dynamic performance evaluation. An increasing trend in academic output has underscored the critical importance of this evolving field since 2022. We support its research by identifying open issues and highlighting future investigation directions.

INDEX TERMS Anomaly detection, concept drift survey, datasets, DDoS, drift adaptation, drift detection, intrusion detection, network intrusion detection, NIDS.

I. INTRODUCTION

Computer networks serve as the backbone of the modern digital era, facilitating global interconnectivity through a wide range of online services. The development of new applications, the widespread adoption of Internet of Things devices, the deployment of high-speed infrastructures, such as 5G, and the shift in user behavior, continuously transform this ecosystem [1], [2], [3], [4]. These mutations frequently alter the network traffic profile, rendering previously defined baselines by security systems obsolete. This phenomenon is referred to as concept drift (CD). As a result, the environment is characterized by heterogeneous, volatile network traffic [5], [6], [7], [8]. Discriminating between environmental evolution and malicious activities is a vital requirement for automated, near-real-time security solutions.

Autonomous security systems are often built upon pattern recognition models, categorized as machine learning (ML), that presuppose learned data distributions remain consistent over time [9], [10], [11], [12], [13]. This assumption does not reflect the real-world non-stationary nature of network streams. Thus, static machine learning architectures are fundamentally challenged by the emergence of CD, which degrades their predictive performance post-deployment [14],

The associate editor coordinating the review of this manuscript and approving it for publication was Asad Waqar Malik.

© 2026 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

[15], [16]. The unaddressed presence of drift can lead to increased false alarms or to missed detection of zero-day attacks, punishing legitimate users and overlooking anomalies. Given the time-sensitive constraints of modern networks, adaptability to dynamic environments is particularly critical, as delays in responding to novel data patterns can create security vulnerabilities and operational instability [17], [18].

Drift-resilient network intrusion detection systems (NIDS) are increasingly gaining notoriety in research. Shallow learning ensembles, clusterization, and deep learning neural networks are often combined with drift detection and adaptation techniques to implement adaptive NIDS. Examples of anomaly detection strategies comprise adaptive random forests [19], k-nearest neighbors [20], convolutional neural networks [21], long short-term memory [22], and autoencoders [23]. Adaptation to dynamic environments is frequently dependent on labeled data, and can leverage retraining whenever drift is detected [24], periodically [25], or by incrementally adding knowledge to the system [26].

This survey article proposes a comprehensive systematic review of the state of the art in concept-drift-resilient network intrusion detection systems. None of the related works simultaneously addressed benchmark datasets, NIDS modeling, drift detection and adaptation, and dynamic-specific performance metrics. The present study aims to bridge this research gap by discussing these essential topics and exploring their interdependencies. By integrating these dimensions, this review provides a cohesive understanding of the relationship between data quality, algorithmic plasticity, and evaluation rigor. For cybersecurity research, this holistic perspective reveals critical vulnerabilities in current adaptive paradigms, such as the reliance on unrealistic labeling speeds. At the same time, for practitioners, it serves as a technical blueprint for navigating operational trade-offs among detection accuracy, computational overhead, and system stability in live, evolving networks.

Building upon this integrated approach, our work introduces an up-to-date taxonomy of model adaptation techniques and an organization of drift-simulation strategies for static datasets. We further analyze the systems' dependence on ground truth and the forgetting of previously acquired knowledge. Finally, future directions in the field are highlighted as a roadmap to address the identified research gaps.

The contributions of this work are sixfold:

- Present the most used benchmarking datasets and drift simulation strategies for developing and evaluating intrusion detection systems;
- Identify the usage of machine learning strategies in implementing NIDS to inform researchers on the most and least explored strategies;
- List the frequently employed methods for detecting the occurrence of concept drift according to ground truth dependence;
- Introduce an up-to-date taxonomy of drift adaptation techniques, categorizing them according to the NIDS' target updating component;
- Quantify the most popular performance metrics, especially those specific to dynamic environments;
- Identify the gaps in research that warrant further investigation.

The remainder of this survey is organized as follows: Section II reviews the related work; Section III presents theoretical foundation on data stationarity and concept drift; Section IV describes the applied methodology for literature reviewing; Section V discusses the obtained results; Section VI highlights the identified open issues, also indicating the future research directions; and Section VII concludes on our findings.

II. RELATED WORK

This section presents a review of relevant works that survey concept drift in network traffic analysis using machine learning models. We discuss their main characteristics, particular insights, and the inferences they provide. Finally, we highlight how our work builds upon these studies by presenting a systematic review of the state of the art in concept drift handling for network traffic classification and anomaly detection.

Riess [27] surveyed the automated adaptation of machine learning models via metaheuristics in environments with concept drift. In this analysis, 17 studies were qualitatively classified by metaheuristic type, adaptation approach, and the regarded drift patterns. The author highlights that population-based methods, such as Genetic Algorithms and Particle Swarm Optimization, are the most dominant techniques for high-dimensional optimization problems in drift adaptation. The work pinpoints flaws in the reviewed studies, such as the lack of specifying the tested drift types or the reliance solely on accuracy for evaluation, but lacks a deeper discussion of the employed drift detection algorithms, datasets, and metrics.

Agrahari and Singh [28] surveyed data stream mining, focusing on drift detectors and their key traits, advantages, and limitations. The authors categorize drift detection methods into an eightfold taxonomy but provide little discussion of drift-based benchmarks and evaluation metrics. The study highlighted the need to optimize false-positive and false-negative rates, handle multidimensional data, and address class imbalance in continuous-flow environments. Besides, the real-time analysis of unsupervised and semi-supervised data streams was identified as a prevailing challenge.

Suárez-Cetrulo et al. [29] published a literature review on the use of machine learning in continuous data streams, with emphasis on handling recurrent concept drift. The authors surveyed passive and active adaptation strategies, highlighting online ensemble, meta-learning, and clustering approaches. Drift-related performance metrics used to evaluate the detection and adaptation capacity of models

TABLE 1. Related surveys on concept drift in machine learning models.

Reference	Year	Drift Detection	Drift Adaptation	Drift metrics	Intrusion Detection	Benchmark datasets
Riess [27]	2022	✗	✓	✗	✗	✗
Agrahari and Singh [28]	2022	✓	✓	✗	✗	✗
Suárez-Cetrulo et al. [29]	2023	✓	✓	✓	✗	✗
Mahdi et al. [30]	2024	✓	✓	✗	✗	✓
Shyaa et al. [17]	2024	✓	✓	✗	✓	✓
Lukats et al. [31]	2025	✓	✗	✓	✗	✓
Moriano et al. [32]	2025	✓	✓	✗	✓	✗
Abdullahi et al. [33]	2025	✓	✓	✗	✗	✓
Our work	**2026**	✓	✓	✓	✓	✓

were discussed, while the benchmark datasets were briefly described. The study highlights that online incremental learning algorithms incur higher computational costs due to the relearning of older patterns in the presence of recurring drifts. The authors suggest that future research further integrate data stream learning, time-series analysis, transfer learning, and deep learning to develop autonomous adaptive strategies.

Mahdi et al. [30] presented a comprehensive roadmap for adapting to concept drift in data stream mining. The authors categorized drift detection techniques into three families: statistical-, window-, and ensemble-based methods. Statistical methods were highlighted as computationally efficient for abrupt drift, while they often struggle with gradual drift. Conversely, ensemble methods were identified as robust against recurring drift, although they suffer from high memory consumption and complexity. They argued that future research must address the class imbalance problem in drifting streams and develop semi-supervised or unsupervised approaches, as the assumption of immediate label availability rarely holds in the real world. However, the work fails to survey drift-specific metrics and solutions to the intrusion detection task.

Shyaa et al. [17] proposed a comprehensive review of 71 papers on integrating machine learning into intrusion detection systems, with an emphasis on the implications of concept drift and feature drift. This study proposed a conceptual framework to address class imbalance, high-dimensional data, and drift challenges to mitigate the obsolescence of intrusion detection systems in dynamic environments. The absence of models capable of handling both concept and feature drift simultaneously was highlighted. Furthermore, the authors pinpointed the lack of methodologies that address both class imbalance and concept drift in intrusion detection systems. The work overlooks specialized evaluation metrics essential for assessing concept drift within the constraints of dynamic, real-time streams.

Lukats et al. [31] conducted a literature review of unsupervised concept drift detectors for data streams that accounted for ground-truth delayed availability. The authors surveyed 61 articles and provided a detailed examination of 10 algorithms based on architecture, core ideas, and input data constraints. Seven of these algorithms were implemented and benchmarked across 11 real-world data streams, being evaluated with drift-specific metrics. The study also highlights the lack of high-quality annotation of drift occurrences in publicly available datasets, as well as the need for further research on the detection of gradual and incremental drift. The survey's scope is limited to drift detection in data streams, thereby omitting the broader discussion on adaptation strategies and their application to the intrusion detection task.

Moriano et al. [32] conducted a systematic literature review on adaptive anomaly detection in Cyber-Physical Systems. The authors evaluated 65 papers and introduced a taxonomy that categorizes them by attack type, application domain, learning paradigm, and data management strategy. The analysis revealed that while online methods are better suited to tackle concept drift in data streams, the majority of existing research still relies on offline, static training. It is also pointed out that most studies address either real-time data processing or model adaptation strategies, but rarely integrate both into a fully adaptive solution. The work lacks an in-depth review of drift-specific evaluation metrics and benchmark datasets.

Abdullahi et al. [33] presented a systematic literature review on concept drift mitigation specifically within time-series applications. The authors analyzed 60 articles, categorizing them based on drift speed and learning paradigm. This work identified support vector machines (SVM) and long short-term memory networks as the most frequently employed models, while pointing to SVM as the most effective for drift detection and adaptation. Furthermore, the authors noted reliance on synthetic datasets, which may not fully capture the complexity of real-world noise. They concluded that future work must prioritize adaptive models that do not require explicit retraining, emphasizing hybrid approaches to better differentiate between actual drift and outliers. The work failed to survey metrics for measuring drift and strategies for implementing intrusion detection systems.

The reviewed related works are summarized in Table 1, where the columns correspond to fundamental concerns for the effective development of drift-resilient traffic analysis. Most studies analyze both detection and adaptation techniques for concept drift. Only a few works review specific drift metrics or focus on intrusion detection. None of the analyzed studies simultaneously covers the topics of concept drift detection, adaptation, dynamic-environment metrics,

intrusion detection, and benchmark datasets. We bridge this gap by conducting an updated, in-depth literature review of the state of the art, comprehending such elementary topics for adaptive NIDS development.

III. BACKGROUND

A. DATA STATIONARITY

Machine learning algorithms are commonly based on the assumption that the training and test data are independent and identically distributed (i.i.d.), and likely stationary over time [34]. Some ML practitioners might assume that each sample in the dataset was generated in isolation without influence from previous samples. They also presume that each training and test data sample is drawn from a single probability distribution, i.e., the samples were generated using the same set of latent rules and thus share similar numerical patterns and trends [35]. The i.i.d. assumption enables developers to train ML models to perform a target task and estimate their runtime generalization performance on a test set before deploying them in a real-world application.

Network anomaly detection has been widely conducted utilizing machine learning, and inherently, the i.i.d. assumption [11], [36]. Computer networks are known for their high dynamism and evolution over time, resulting in non-stationary traffic trends [37], [38], [39]. Thus, ML-based NIDS built upon the i.i.d. data paradigm will likely suffer from long-term performance degradation when deployed to process real-time data, since the traffic probability distribution is expected to shift over time. Realistic NIDS implementations must account for data non-stationarity and integrate mechanisms to detect and adapt to drift in the traffic probability distribution, thereby promoting continuous effectiveness and generalization.

B. CONCEPT DRIFT

Concept drift is a subfield of statistics and machine learning that investigates emerging patterns, trends, and relationships in data over time [40], [41]. The formal mathematical definition of CD is commonly stated in a supervised learning setting. Let $x \in \chi$ denote a feature vector sampled from feature space and $y \in Y$ its respective classification label, where $Y = \{class_1, class_2, \ldots, class_k\}$. The dataset used to develop an ML model consists of (x, y) sequential data points. Concept drift is characterized by a change in the joint probability distribution of such points between distinct timestamps t and $t + n$, written as $P_t(x, y) \neq P_{t+n}(x, y)$ [42]. Following Bayes' theorem, $P_t(x, y)$ can be factored as $P_t(y|x) \times P_t(x)$ [14]. Therefore, a change in $P_t(x, y)$ is directly caused by an alteration in the label posterior distribution $P_t(y|x)$, or in the feature distribution $P_t(x)$, or both.

CD can be classified according to which factor of $P_t(x, y)$ triggers the data shift [17], as shown in Fig. 1. If the variation is caused by $P_t(x) \neq P_{t+n}(x)$ while $P_t(y|x) = P_{t+n}(y|x)$, then it is named virtual drift or feature drift (b). The term ''virtual'' indicates that, despite a change in the distribution of feature vectors, the relationship between the input and output $P_t(y|x)$ remains unchanged. In such a scenario, the classification model may not need to be updated, as its learned decision boundary continues to separate samples from different classes correctly. If the change is caused by $P_t(y|x) \neq P_{t+n}(y|x)$ whilst $P_t(x) = P_{t+n}(x)$, then it is called a real drift (c). The word ''real'' means the latent function, which maps inputs to outputs, has changed. In this case, the model must be updated to track the new decision boundary to classify feature vectors accurately. In the hybrid case, both $P_t(y|x)$ and $P_t(x)$ change, also requiring model updates to maintain classification accuracy (d).

Concept drift is additionally categorized by how the old data distribution $P_t(x, y)$ transitions to the novel one $P_{t+n}(x, y)$ over time, as illustrated in Fig. 2 [43]. For example, sudden concept drift is characterized by an abrupt change in distribution between two consecutive time steps (a). Such unexpected drift necessitates prompt model adaptation to minimize the harm caused by false inferences. Conversely, drifts may also occur gradually, with the old distribution smoothly transitioning to the new one over multiple time steps (b). Since such drift is more subtle, it may be more complex to detect than the sudden counterpart. Recurrent drifts constitute a challenging category in which different data distributions cyclically and indefinitely alternate throughout time (c). In such a scenario, models should retain knowledge of all distributions and make predictions based on the current distribution in the cycle. Concept drift can occur under various conditions, making it difficult to find a single universal method for detecting it and adapting the learning model. Hence, it is evident that CD handling follows the No Free Lunch theorem. Researchers should develop specialized detection and adaptation methods tailored to the drifts they expect to encounter in their respective applications [44].

IV. METHODOLOGY

Our research goal is to conduct an empirical review of the state of the art in ML-based anomaly and intrusion detection systems that are robust to concept drift. A generic representation of the common pipeline involving the development of concept drift-resilient NIDS is depicted in Fig. 3. This structure frequently encompasses network traffic collection via data streams, knowledge integration, network intrusion detection systems and their inferences, drift detection, and integration of drifted data. We survey such common steps during the development of NIDS by addressing the following research questions:

- **RQ1** - What datasets are considered for benchmarking experiments?
- **RQ2** - Which machine learning models have been implemented to address anomaly detection?
- **RQ3** - What algorithms are predominantly utilized for the identification of concept drift?

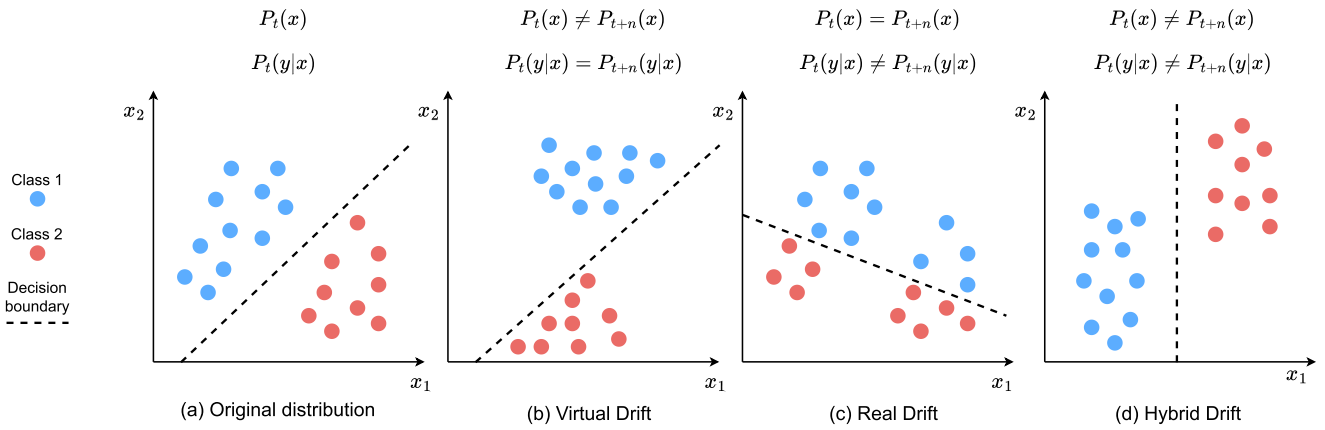


FIGURE 1. Concept drift trigger types: no drift (a), virtual drift (b), real drift (c), and hybrid drift (d).

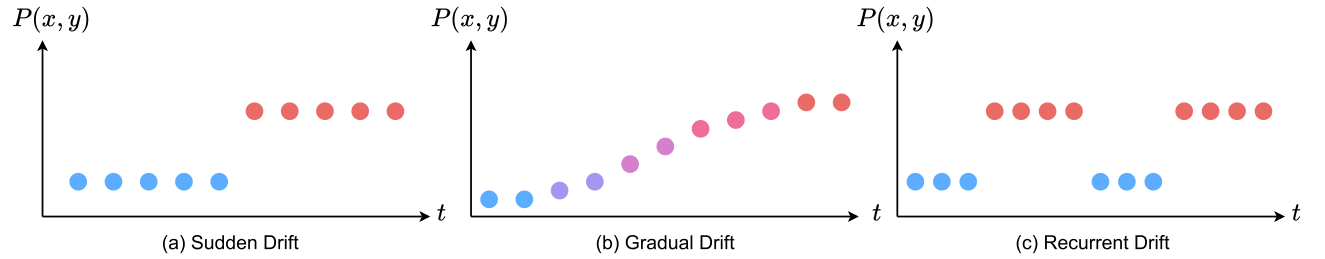


FIGURE 2. Concept drift transition types: sudden (a), gradual (b), and recurrent (c).

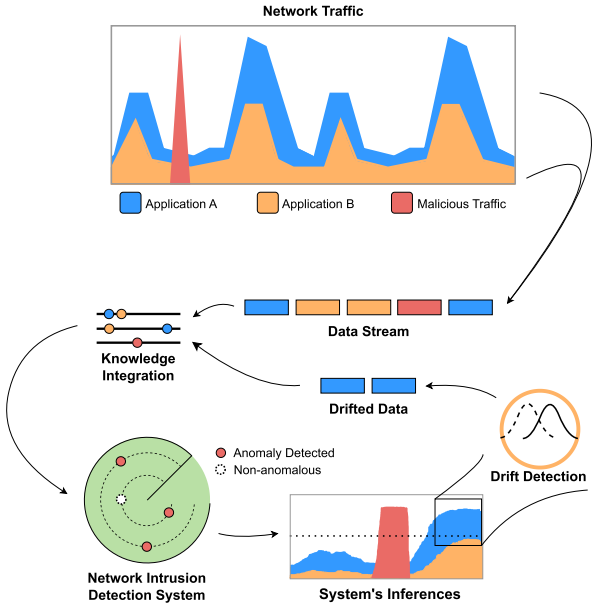


FIGURE 3. The common pipeline for implementing drift-resilient NIDS.

- **RQ4** - How are adaptation techniques structured to mitigate the impact of concept drift on model performance?
- **RQ5** - Which performance metrics are employed for evaluating both intrusion and drift detection?
- **RQ6** - What are the current open issues and prospective avenues for future investigation in this field?

Fig. 4 depicts the research methodology's sequential approach to the proposed empirical literature review and the resolution of the aforementioned questions. The first stage in conducting a comprehensive review of this field involves the identification and selection of appropriate scholarly databases (Fig. 4a Database Selection). We defined some specific criteria for selecting appropriate databases for our study: online accessibility through our institutions' agreements; indexing of high-impact journals aligned with the research scope; a predominance of English language publications; support for advanced Boolean logic and metadata filtering; and the capability to export results in standardized formats such as.csv or.bib. ACM Digital Library, IEEE Xplore, ScienceDirect, and Springer Link were the four databases found to satisfy these specifications.

The search scope was defined using Boolean query strings incorporating critical domain-specific keywords (Fig. 4b Identification). To ensure a comprehensive capture of relevant literature, we built a query string with 12 related terms, namely: Intrusion Detection, Anomaly Detection, Attack Detection, Traffic Classification, IDS, Network Security, DDoS, Concept Drift, Data Drift, Real Drift, Virtual Drift, and Feature Drift. Since the selected databases' search engines support a maximum of 8 Boolean operators, we reformulated the query into three distinct ones:

- **Q1**: ("Intrusion" OR "Anomaly" OR "Attack") AND "Detection" AND "Concept Drift";

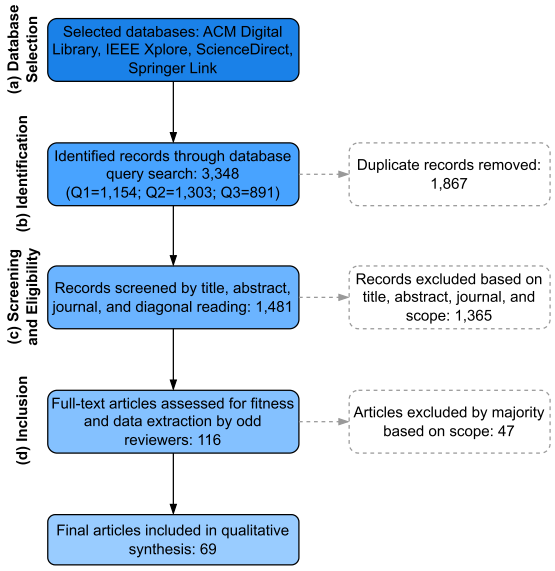


FIGURE 4. Research methodology.

- **Q2**: (“Intrusion Detection” OR “Traffic Classification” OR “IDS” OR “Network Security”) AND (“Concept Drift” OR “Data Drift” OR “Real Drift” OR “Virtual Drift” OR “Feature Drift”);
- **Q3**: “DDoS” AND “Concept Drift” OR “Anomaly Detection” AND “Concept Drift” OR “Intrusion Detection” AND “Concept Drift” OR “Traffic Classification” AND “Concept Drift”.

As shown in Fig. 4, Q1 yielded 1,154 papers, while Q2 and Q3 returned 1,303 and 891, respectively. Only full-length journal articles published between 2022 and 2025 inclusively were considered, which summed up to 3,348 records across the selected databases and queries. While it is acknowledged that significant foundational work exists prior to 2022, focusing on full-length articles from this period onward ensures that our review reflects the most recent advancements and emerging challenges in this rapidly evolving field. This temporal scope also preserves analytical depth, enabling rigorous screening of the high volume of records generated by our comprehensive search strategy.

After removing 1,867 duplicates resulting from overlapping queries in the identification phase, the remaining 1,481 articles underwent a manual selection process to assess their alignment with our research scope (Fig. 4c Screening and Eligibility). We check whether the papers proposed intrusion or anomaly detection solutions for computer network environments that specifically addressed concept drift. Furthermore, surveys and review articles were excluded to focus on primary research contributions. This manual refinement is essential, as keyword matches do not always guarantee thematic relevance. Specific pertinent works from other domains were included if they utilized computer network datasets and offered innovative approaches adaptable to the network security paradigm.

After data cleansing, the screened 116 full-text articles were thoroughly assessed for fitness and suitability for data extraction, answering the six research questions defined beforehand (Fig. 4d Inclusion). We divided the research questions into two scopes: intrusion detection (RQ1, RQ2, and RQ5) and drift handling (RQ3 and RQ4). To mitigate bias and ensure a robust analysis, each scope was assigned to an odd number of co-authors who reviewed all screened papers. Each author was responsible for analyzing and tabulating the RQs within their assigned scope, subsequently contributing to the writing of the sections corresponding to their findings. This setup guaranteed both overlap in analysis (multiple perspectives per RQ) and a built-in tiebreaker criterion for inclusion decisions (odd decisions per article, which inherently resolves reviewers’ disagreements through majority voting).

The reading overlap played a vital role in our methodology, avoiding biased analysis and contributing to a fairer process, while open issues (RQ6) were noted by every author throughout the inclusion task and compiled by the first author. The insights extracted from these works are used to delineate the state of the art, with the following sections organized to address each research question separately. This structure allows readers to navigate the review efficiently and focus on specific areas of interest. The results of the conducted survey are exhibited hereafter.

V. RESULTS

The state of the art on concept drift handling in intrusion detection is reviewed in this section. The following subsections answer the six proposed research questions by presenting the latest advancements regarding benchmarking datasets, machine learning-based intrusion detection, concept drift detection and adaptation, and performance metrics.

A. METADATA

Firstly, we analyze the metadata associated with the articles gathered through our research methodology. We conducted data collection on August 5th, 2025 across four data sources: IEEE Xplore, Science Direct, ACM Digital Library, and Springer. These queries produced an initial set of 1481 works, with Science Direct being the primary source (823), followed by Springer (372), IEEE Xplore (181), and ACM Digital Library (105).

We then filtered the articles, discarding those that did not meet our inclusion criteria. This process narrowed the final set to 69 papers, representing 4.7% of the original retrieval. Such a reduction suggests that concept drift is widely acknowledged in the field of intrusion detection, but solutions to this problem are scarce.

Fig. 5 illustrates the annual distribution of selected works, revealing a consistent upward trend from 2022 to 2025. This rise in scientific production underscores that concept drift remains an evolving and active research area. It is worth noting that, although six early access articles scheduled for

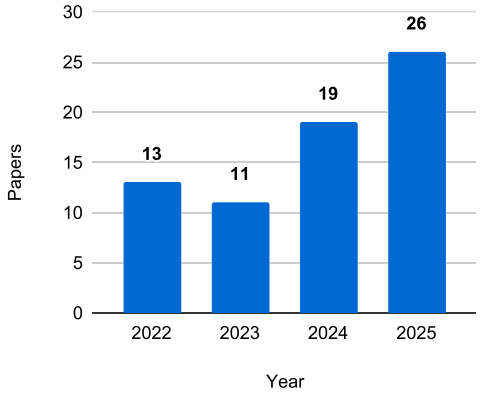


FIGURE 5. Selected articles distribution per year.

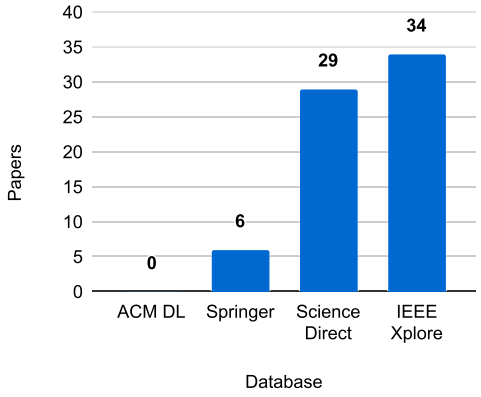


FIGURE 6. Selected articles per database.

2026 were screened, none met the eligibility criteria. The distribution of selected papers per data source is illustrated in Fig. 6. IEEE Xplore and Science Direct collectively account for over 90% of the selected studies, suggesting they are primary repositories for adaptive NIDS research. Fig. 7 depicts the distribution of articles per journal. This distribution highlights the most active periodicals, helping researchers target relevant venues. For visual clarity, journals contributing two or fewer articles were omitted from this figure.

B. DATASETS

In this section, we answer the first research question (RQ1), providing more details about the datasets used in the reviewed works. Such data span multiple execution scenarios, including computer networks, forecasting, power stations, LED displays, cluster problems, system calls, mobile Android API calls, physical activity, quality of service, and server performance. Given the scope of this survey, only network-related datasets were considered, yielding a total of 50 items. Those that inherently present drift are listed in Table 2, which displays their names and surveyed works that employ them; the 10 most used static datasets are exhibited in Table 3, which shows their names and times used across the analyzed papers.

TABLE 2. Public drift-based datasets and their presence in surveyed works.

#	Dataset	Works
1	WIDE - MAWI	[45], [46]
2	DS2OS Traffic Traces	[47]
3	IP maliciousness	[48]
4	UGR'16	[49]

TABLE 3. Public static datasets and their number of appearances.

#	Dataset	Times used
1	CIC-IDS2017	17
2	KDDCup 1999	14
3	NSL-KDD	11
4	CSE-CIC-IDS2018	9
5	IoTID20	8
6	UNSW-NB15	7
7	IoT-23	4
8	BoT-IoT	3
9	CTU-13	3
10	HIKARI2021, and ISCXVPN2016	3

As presented in Table 2, only 4 datasets showed any type of drift. The MAWI Working Group archive presented in [50] is the most used drift-based dataset, appearing in 2 studies: dos Santos et al. [45], and H. Wu et al. [46]. This dataset spans over two decades of traffic from the WIDE Project, providing a source for the analysis of gradual drift resulting from long-term evolution in network protocols and user behavior. The DS2OS Traffic Traces dataset is employed by Wahab [47], and can be found in [51], exhibiting drift within benign IoT traffic patterns; the specific type of this drift remains undocumented by the original authors. The IP Maliciousness dataset [52] is considered in García and DeCastro-García [48] and was curated by the Spanish National Institute of Cybersecurity; it captures a longitudinal shift by providing data from two distinct periods (May 2021 and April 2022), with an eleven-month hiatus between captures, facilitating the study of drift. The UGR'16 dataset [39], present in Shajari et al. [49], addresses cyclostationarity, capturing the periodic and seasonal fluctuations inherent in real-world large-scale ISP traffic, helping to identify benign cyclical changes in IDS.

As shown in Table 3, a small group of static datasets prevails in the surveyed literature. CIC-IDS2017 [53] is the most prevalent, with 17 appearances, followed by KDDCup 1999 [54] and NSL-KDD [55] with 14 and 11 occurrences, respectively. The remaining most frequent datasets each appeared at least three times, with HIKARI2021 and ISCXVPN2016 sharing the tenth position. Beyond these, seven datasets were featured in only two studies each, while 27 others appeared in only one study each.

The scarcity of concept drift datasets hinders the benchmarking of drift-resilient NIDS. To reduce such a problem, researchers commonly apply data preprocessing techniques to static benchmarks aiming to simulate non-stationary testing environments, as detailed in Table 4. These processing methods are categorized as follows:

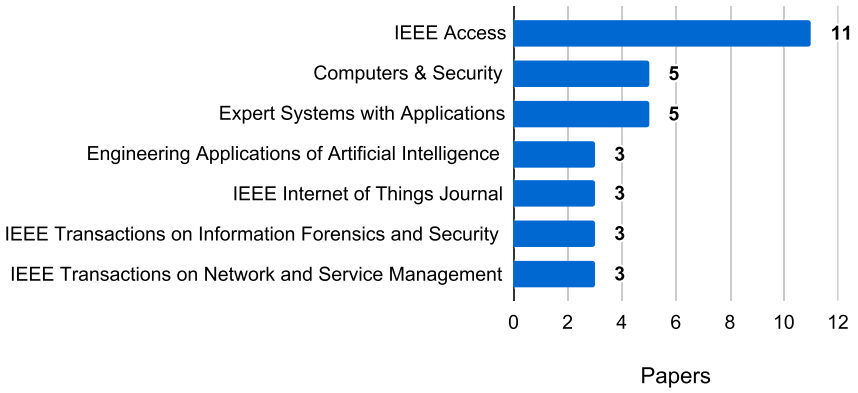


FIGURE 7. Selected articles distribution per journal.

- **Class imbalance**: this approach involves generating discrete subsets with varying percentages of specific traffic types. By artificially shifting the class proportions across different temporal windows, authors can evaluate an IDS's robustness against virtual drift. While effective for testing sensitivity to distribution changes, this method often fails to simulate changes in the underlying feature relationships, potentially leading to an incomplete evaluation of detection performance in complex environments. For example, Qiao et al. [22], using BoT-IoT; M. Liu et al. [56], with MIRAGE2019, ISCXVPN2016, and a proper (NJUPT2023); and Liao et al. [57], utilizing KDDCup 1999.
- **Data fusion**: this method involves merging traffic data from different periods to create a heterogeneous stream. These samples can originate from the same source, following the example of Niu et al. [26], that merged different days from NSL-KDD; or disparate generations, as done by Zha et al. [18], who merged CIC-IDS2017 and CSE-CIC-IDS2018, and Huang and Ma [58] that used VeReMi and VeReMi Extension. This strategy helps to simulate the long-term evolution of network behaviors. The primary limitation of data fusion is the potential for architectural inconsistencies and feature-set discrepancies across datasets, which may introduce artifacts that do not reflect genuine network behavior evolution.
- **Synthetic generation and injection**: this category encompasses both the modification of real datasets and the use of purely synthetic benchmarks. Researchers use frameworks like Binary Generator with Feature Drift (BG-FD) [59] or Massive Online Analysis (MOA) [60], [61] to create controlled drift based on pre-existing network traces. Additionally, foundational synthetic generators present in the MOA toolkit [60], such as Streaming nsemble Algorithm (SEA), Stagger, Agrawal, Radial Basis Function (RBF), Waveform, and Hyperplane, are frequently employed as baseline benchmarks to validate detection algorithms in highly controlled, albeit abstract, non-stationary environments. These generators often lack the high-dimensional complexity and semantic nuances of real-world environments, which can result in models that perform well on benchmarks but struggle with the unpredictability of live deployments.
- **Geometric partitioning**: this technique utilizes mathematical distances, specifically cosine distance, to segment data samples. By partitioning the feature space based on geometric dissimilarity, researchers can simulate apparent conceptual shifts between different segments of the data stream, as implemented by Junjie Zhang et al. [25], which used the cosine distance in the ACI-IoT-2023 dataset. Relying solely on geometric distance may overlook temporal dependencies and structural patterns inherent to real-world behaviors, potentially creating artificial boundaries that do not reflect realistic shifts.

Two other strategies were identified among the surveyed research to validate systems' resilience to drift. The first strategy is employed by Qiao et al. [22], which decoupled their drift detection method from the NIDS. While the intrusion detection capabilities of their system are assessed using the BoT-IoT dataset, an IoT-based network traffic collection, the drift detection module was evaluated separately on 2 other datasets. These datasets, namely SEA Concept and UG-2C-5D, comprise domain-agnostic synthetic data streams that simulate abrupt and gradual drift on generic numerical data, respectively. This approach allows for an isolated performance assessment of drift detection in scenarios where no datasets with drift are available within the scope of the work. The second approach is present in the works [41], [57], [60] and is related to the assessment of unseen outliers in data stream classification. This strategy groups new outliers by similarity, considering a lot of related occurrences as a new class. Emerging classes can comprise drifted benign traffic or zero-day attacks, which may not be suitable for most intrusion detection implementations. This methodology risks increasing false-positive and false-negative rates, as the

TABLE 4. Main drift generation methods.

Category	Description	Works
Class imbalance	Make subsets with different class proportions	[22], [56], [57]
Data fusion	Merge similar datasets	[18], [26], [58]
Synthetic generation and injection	Use generators or tools to simulate/inject drift	[59], [60], [61]
Geometric partitioning	Separate data samples based on distance	[25]

model may not distinguish between legitimate concept evolution and malicious activity.

Researchers generally adopt one of four primary strategies to address the intersection of computer networks, intrusion detection (or traffic classification), and concept drift under limited data resources:

1) The first approach utilizes specialized datasets that inherently contain concept drift; while this is the most direct methodology, its application is significantly constrained by the scarcity of such input.
2) The second strategy involves applying the synthetic generation techniques outlined in Table 4 to modify static datasets and simulate non-stationary behaviors.
3) A third option is to treat new patterns as emerging classes. This approach is related to the traffic classification task and may not be suitable for intrusion detection, as new classes can represent drifted benign concepts or zero-day attacks. The lack of differentiation between legitimate and anomalous traffic may increase false alerts.
4) Finally, the most prevalent strategy involves implementing detection systems that, in theory, mitigate concept drift; however, these systems often lack rigorous validation through specialized testing. The absence of such targeted evaluation fails to empirically verify whether the proposed solution can maintain predictive accuracy over time in dynamic, real-world environments.

C. NETWORK INTRUSION DETECTION SYSTEM

In this section, we answer the second research question (RQ2) by organizing the collected NIDS models according to their ground-truth dependency. Network intrusion detection systems have numerous implementations. They range from signature to anomaly-based methods, grounded in behavioral patterns and statistical analysis, respectively. Both strategies require the extraction of intricate data structures, favoring machine learning implementations, which emerge as the predominant approach among surveyed works. ML models can be categorized into three main learning paradigms: supervised (59% of compiled research), semi-supervised (26%), and unsupervised (15%). These are further divided into a range of subcategories, as shown in Fig. 8. The distribution of ML strategies for NIDS implementation across supervised, semi-supervised, and unsupervised paradigms is depicted in Fig. 9a, 9b, and 9c, respectively.

A chronological analysis reveals an upward trajectory in semi-supervised research. The volume of publications

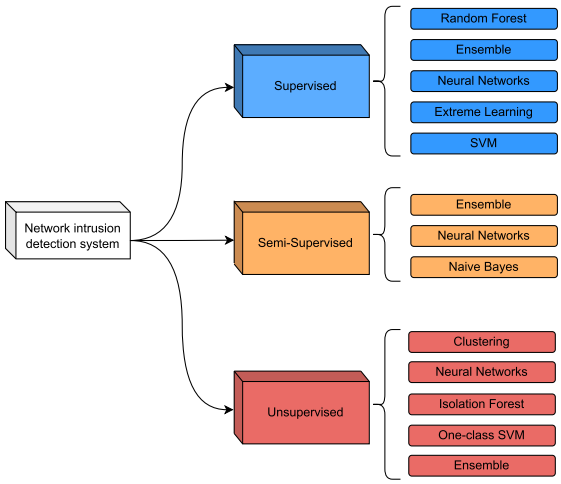


FIGURE 8. Network intrusion detection algorithms taxonomy.

grew from two identified works in 2022 [41], [62] to seven by 2025 [20], [56], [58], [60], [63], [64], [65], representing a proportional increase from around 15.4% to 20.6% of the total research output. This progression underscores a heightened scholarly focus on mitigating the label-scarcity challenges inherent in real-world networks. Parallel to this trend, shallow individual classifiers, such as SVMs, Naive Bayes, and clustering, have declined from three works in 2022 [66], [67], [68] to a single instance in 2025 [60]. Conversely, ensemble techniques and neural network architectures, including random forest, isolation forest, and extreme learning, have seen prolific growth, rising collectively from 9 to 25 works over the same period. Summing up, there is an increasing interest in the semi-supervised paradigm, as well as ensemble and neural network techniques, while the use of shallow learning is decreasing. A breakdown of these algorithms by paradigm is provided in Table 5.

1) SUPERVISED LEARNING

Supervised learning employs labeled data during training to map input features to target variables, encompassing both differentiation of categorical outcomes and estimation of continuous values. In the network intrusion detection context, traffic can be organized into binary classes (i.e., legitimate and fraudulent) or into multiclass schemes that comprise a variety of specific attack types. Among the surveyed literature, data were most commonly available in streams and IP flow statistics, while the employed anomaly detection

TABLE 5. Network intrusion detection algorithms. SP: Supervised, US: Unsupervised, SS: Semi-supervised.

#	Algorithm	Paradigm	Works
1	Neural Networks	SP	[18], [21], [22], [25], [42], [45], [47], [69], [70], [71], [72], [73]
2	Random Forest	SP	[19], [26], [48], [74], [75], [76], [77], [78], [79], [80], [81], [82]
3	Ensemble	SP	[61], [83], [84], [85], [86], [87], [88], [89], [90], [91], [92]
4	Neural Networks	SS	[23], [24], [37], [41], [56], [63], [93], [94], [95]
5	Ensemble	SS	[20], [57], [58], [62], [64], [65], [96], [97]
6	Extreme Learning	SP	[59], [98], [99], [100]
7	Clustering	US	[46], [67], [68]
8	Neural Networks	US	[49], [101], [102]
9	Isolation Forest	US	[103], [104]
10	SVM	SP	[66], [105]
11	Ensemble	US	[106]
12	Naive Bayes	SS	[60]
13	One-class SVM	US	[107]

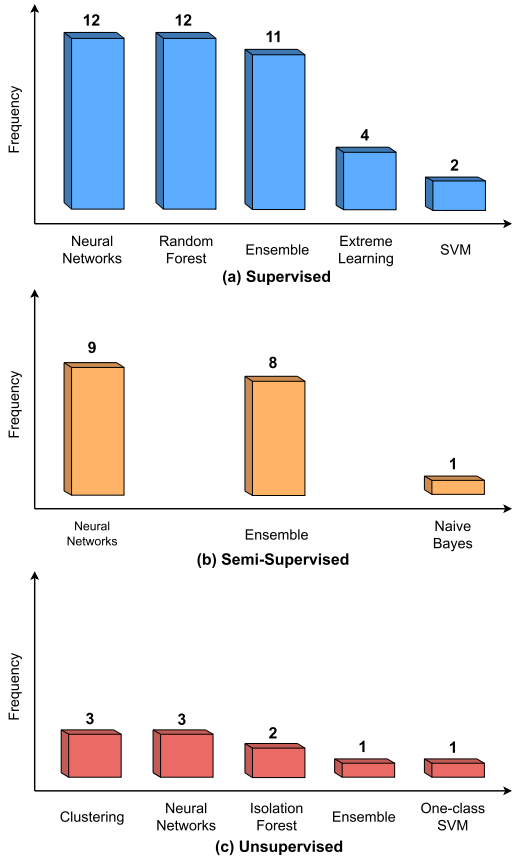


FIGURE 9. Network intrusion detection algorithms distribution.

methods ranged from traditional ML, such as Random Forest and SVM, to Neural Networks, as displayed in Fig. 8.

Decision tree algorithms represent the majority of the gathered methods, being combined into Random Forest (RF) ensembles in 29% of supervised works, as depicted in Fig. 9a. Random forests leverage ensemble learning to create multiple decision trees that are trained on different random subsets of data instances and features. This strategy, called bootstrap aggregating (Bagging), favors parallelism, reduced model variance, and enhanced robustness. A voting procedure aggregates each base classifier's decision into a single ensemble inference. Concept drift may significantly affect the performance of this type of algorithm when it models the entire dataset offline and statically; thus, it is typically implemented in its "adaptive" form, called Adaptive Random Forest (ARF), where each tree in the ensemble is a Hoeffding tree. Hoeffding trees do not require all the data to be available at once, so they can be trained continuously. Paim and Enembreck [19] used an adaptive tree ensemble with an algorithm to select which base classifiers are considered in the voting. Eldhai et al. [74] compared adaptive random forests with Hoeffding adaptive trees. Niu et al. [75] use an ensemble of trees that creates new classifiers when it detects that one has a low performance. Abdulboriy and Shin [76] created an ensemble that includes ARF, k-nearest neighbors, and a softmax regressor. Wu et al. [77] use ARF-EDDM to adapt to drift when detected by other methods.

Other ensemble methods appeared in roughly 27% of the supervised reviewed NIDS. The ensemble's composition varies across works, since any combination of base classifiers can form a committee. Some works, such as Malathy et al. [83], use the divergence of classifiers' inferences to detect drifting over time. Jiang et al. [84] use an ensemble of light gradient boosting machines that attribute voting weights to the base learners based on their performance. Alashhab et al. [85] propose an ensemble of multi-layer perceptron, BernoulliNB, passive-aggressive, and stochastic gradient descent classifier. Shyaa et al. [86] presented a genetic algorithm to create an ensemble of extreme learning machines variants. F.-Q. Li et al. [87] employed AdaBoost to adjust the weights of the data for their decision tree ensemble.

Neural networks (NN) accounted for 29% of label-dependent anomaly detection systems, and work by combining neurons into sequential layers. These algorithms feed data forward and harness backpropagation to adjust the neurons' weights during training. In supervised learning, they are often

trained to map feature inputs directly to the data classes. Model size may vary based on data dimensionality and availability to avoid overfitting and underfitting. Since they do not offer a clear explanation of why or how each value contributes to each inference, explainable AI algorithms have been employed to provide deeper insights into the decision-making process [21], [37], [65], [73]. Over the years, multiple variations of neural networks have been developed to improve their performance across different tasks. For instance, convolutional neural networks (CNN) specialize in computer vision tasks where inputs do not represent specific features; instead, they extract spatial relationships from data. Cerasuolo et al. [21] and Junjie Zhang et al. [25] implement CNN to classify networking data.

Another specialized NN found in the literature is the long short-term memory (LSTM) network. This type of model has a memory module to store a state that affects future predictions, enabling sequential processing and the extraction of temporal context from data. These networks are especially useful for anomaly detection, since flows often exhibit temporal correlation. Qiao et al. [22] compare the performances of CNN and LSTM in multiclass classification on a drifting environment. Soltani et al. [69] present a similar approach by comparing CNN and LSTM within a framework that continuously adapts to data drift.

The extreme learning paradigm appeared in almost 10% of the supervised articles investigated, which are listed on Table 5. This neural network comprises a single hidden layer, designed to be simpler and faster than multi-layer methods. Since its training is performed non-iteratively, the model can be updated when necessary in an online learning context. Nitish et al. [98] and Rabash et al. [59] both present online extreme learning methods for updating in a time-sensitive environment for network anomaly detection.

Support vector machines (SVM) represent about 5% of the annotated data learning and do not rely on neurons. This technique determines a hyperplane boundary between the defined classes, aiming to maximize the distance between the established partition and the closest data points from each class. Jain et al. [66] presented an SVM for classifying data processed by a clustering algorithm. Kalita et al. [105] also applied an SVM for binary classification of anomalies.

2) SEMI-SUPERVISED AND HYBRID

The classification of semi-supervised algorithms remains diverse, largely due to the different definitions within the literature. While Gu et al. [20] maintain the traditional view by utilizing a limited set of labeled data alongside a larger unlabeled set, other perspectives expand this scope significantly. For instance, Pasikhan et al. [97] categorize single-class training as semi-supervised, while Liao et al. [57] include algorithms capable of incorporating entirely new data classes. This broad conceptualization allows for a more permissive inclusion of methodologies, though it risks overlapping with adjacent learning paradigms. The surveyed semi-supervised implementations comprise Neural Networks, Ensemble, and Naive Bayes, as illustrated in Fig. 8. Ensembles accounted for 44% of the reviewed semi-supervised studies, while neural networks accounted for 50%, as compared in Fig. 9b.

One example of semi-supervised ensemble-based systems is presented by B. Li et al. [62]. The authors propose an ensemble that gradually grows to form clusters of data labeled by a specialist and can detect new classes of attacks. An isolation-forest-inspired ensemble was proposed by Zhao et al. [96] that uses labeled data during training but can still adapt to unknown classes. A third ensemble was presented by Gu et al. [20], which combines a fuzzy inference system with a k-nearest neighbors (KNN) ensemble and a challenging sample learner. This system has a supervised step to build a knowledge base, followed by semi-supervised learning via pseudo-labeling with a KNN ensemble to improve anomaly detection continuously.

Semi-supervised NIDS based on neural networks are often trained for regression tasks to either measure anomaly-prediction error or generate a deviation score relative to a benign base class. For instance, Wang et al. [93] propose an LSTM-based framework that performs one-step-ahead predictions within a sliding window. Anomalies are identified by modeling prediction errors through a Gaussian distribution, where an instance is flagged if its calculated anomaly score exceeds a predefined threshold. However, an arbitrary number is chosen as this limit, which requires labeled data. Autoencoders were also employed, compressing and decompressing data, ultimately measuring the decompression error. Faber et al. [23] declare as anomalous any data that makes its autoencoder's error surpass the defined threshold, which is defined by the anomalies available in the datasets using a ROC Curve. A different autoencoder approach was employed by Yin et al. [63], which uses supervised learning to guide the training of a contrastive model that maximizes the distance between instances within each class. After deployment, the model is capable of working unsupervisedly.

3) UNSUPERVISED

Machine learning models are classified as unsupervised if their training and deployment completely forgo the use of labeled data. Unlike supervised-learning algorithms that leverage a function that maps a set of entries to a desired output, unsupervised models identify the inherent structure of data to determine the patterns that emerge across different instances. Since these methods can be designed to recognize a general base trend in data and alert when a deviation occurs, they are frequently employed for network intrusion detection. Accordingly, concept drift may primarily increase the false positive rate, as once drifted legitimate traffic can be mistaken for attacks. The unsupervised methods extracted from the analyzed works span over Clustering, Neural Networks, Isolation Forest, Ensemble, and One-class SVM, as listed in Table 5.

Clustering algorithms group data in the feature space based on their similarities, accounting for 30% of unsupervised approaches, as portrayed in Fig. 9c. Among the observed literature, two main categories were found: agglomerative and density-based. Agglomerative clustering works by treating each data point as its own cluster, then grouping them into larger clusters based on proximity. This method has an advantage over more common methods like k-means, since it does not require a predefined number of clusters as a hyperparameter; it works similarly to a taxonomy tree, where the number of clusters is determined by the height at which the dendrogram is thresholded. This method is often called hierarchical, as in Schwengber et al. [68], which uses Euclidean distance to calculate the difference between network flows and iteratively groups the two closest clusters. Another agglomerative approach is presented by Wu et al. [46], combined with a divide-and-conquer approach. To handle bulkier data volumes, the model divides traffic feature vectors into multiple blocks, creates clusters for each block, and then merges the results. Conversely, density-based algorithms summarize data into micro-clusters that may grow or shrink over time, as presented by Scaranti et al. [67]. These clusters are categorized by density: high-density ones represent normal behavior, intermediate ones might represent anomalies or evolve into high-density normal ones, and sparse ones are composed of outliers.

Neural networks can employ unsupervised learning using prediction or reconstruction error, comprising 30% of surveyed label-independent intrusion detection systems. By employing a symmetrical architecture that progressively constrains and subsequently expands layer dimensionality, autoencoders induce the extraction of salient features through lossy compression into a latent representation, followed by an output reconstruction phase. The error obtained by comparing a reconstructed sample to its original counterpart can be harnessed to separate normal and abnormal traffic. If the reconstruction error for a specific instance surpasses a threshold, it is considered an anomaly. Unsupervised NIDS usually define such threshold independently of labeled data, as done by Shajari et al. [49], using the ceiling function of the mean squared error observed during training. Another approach is presented by Tang et al. [101], which uses extreme value theory based on the distribution of anomaly scores in a data stream to determine the threshold.

A different unsupervised algorithm class is the isolation forest, which appears in 20% of this learning paradigm's research. Designed for anomaly detection, this method operates under the principle that normal traffic is predominant and anomalies are different and rare. An isolation forest works by randomly selecting a feature and a split value. As separations are performed, samples are progressively isolated, yielding distinct classifications for samples based on the number of splits it takes to isolate them. The fewer splits required to isolate the data, the more anomalous it is considered. Mrudula et al. [103] propose a system that integrates isolation forests to detect anomalies and incorporates decision trees to classify them. Similarly, Elsaid and Binbusayyis [104] also presume data is mostly normal, so the isolation forest algorithm is capable of separating outliers from normal traffic.

D. DRIFT DETECTION

In this section, we discuss the summary of the drift detection algorithms found in the selected works, answering the third research question (RQ3). Fig. 10 is a taxonomy of how these solutions are categorized according to their implementation approach. Supervised drift detection refers to algorithms that identify concept deviations by calculating some error metric on labeled data. If the error exceeds a predefined threshold, the algorithm infers that the accuracy drop is due to concept drift within the analysis window [24], [89]. Unsupervised drift detection refers to algorithms that recognize concept alteration by analyzing changes in the statistical distributions of unlabeled data. These usually compute a divergence metric between historical data and a recent window, inferring drift from the magnitude of the divergence [60], [73], [103]. Semi-supervised drift detection algorithms present a hybrid approach that leverages the strengths of using labeled and unlabeled data [77], [81], [104].

Fig. 11 shows a frequency histogram for the drift detection algorithms implemented by the reviewed works. The unsupervised paradigm is the most frequent, with 23 occurrences. The widespread adoption of this implementation approach stems from its independence from labeled data, which reduces costs in production environments. Unsupervised solutions lower the overall intrusion detection system's response latency, since raw data can be processed directly, avoiding the delay of the labeling procedure [56], [94]. Nevertheless, 22 of the reviewed works still implement the supervised paradigm. Their high accuracy in identifying drifting samples explains their frequent use in concept drift detection despite their inherent drawbacks. The semi-supervised was less commonly observed, appearing on 9 works. Future work may study how to better use the semi-supervised, hybrid approach to detecting concept drift with high accuracy while minimizing computational costs.

We should highlight that 14 of the reviewed works proposed adaptive intrusion detection systems that do not rely on concept drift detection. They present a proactive, passive strategy in which the intrusion detection system is updated recurrently with newly collected data without depending on a concept drift trigger. For instance, Rabash et al. [59] assume that drift is a recurring problem and execute the Enhanced Dynamic Filter-Based Feature Selection and classifier model retraining at regular time intervals. Liu et al. [82] proposed an online active learning IDS that outputs confidence values for each inference on network traffic and regularly retrains the model using samples with low prediction confidence. Yin et al. [63] discussed how to apply contrastive autoencoders to detect anomalous network traffic, which is adaptable to concept drift through periodic retraining using historical and recent pseudo-labeled traffic samples.

The scientific community has recently been exploring a multitude of drift detection algorithms based on different paradigm implementations. Table 6 lists the 32 drift detection algorithms identified, indicating the implementation paradigm and the corresponding works. The most common strategy for detecting drift is to track the increase in the system's error rate between past and current timestamp analysis. Such detection algorithms occurred 8 times in the reviewed works and are classified as supervised, since the applied error metrics compare the predicted labels with the corresponding ground-truth labels. Another frequently used algorithm is adaptive windowing (ADWIN), a flexible method applied across the three considered paradigms of drift detection: supervised, unsupervised, and semi-supervised. The drift detection method (DDM) and the early drift detection method (EDDM) are well-established algorithms that are frequently applied in isolation or in combination with ADWIN to detect drift in a supervised or semi-supervised fashion, respectively. Multiple authors build their unsupervised or semi-supervised drift detection methods by calculating a divergence metric between the statistical distributions of historical and recent data windows. The applied ones include, but are not limited to, the Kolmogorov-Smirnov, Kullback-Leibler, Entropy, Page-Hinkley, Jensen-Shannon, and Wasserstein metrics. The following subsections provide a further discussion on the most common drift detection algorithms.

1) SUPERVISED

The drift detection algorithms are classified as supervised when they require labeled data to identify behavior evolution in traffic data. The supervised approach commonly relies on comparing the change in IDS classification error on traffic data between previous and current timestamps. The simplest realization of this method is to track the accuracy metric of traffic classification. For instance, Eskandari and Khotanlou [71] and Saravanan and Balasubramanian [80] proposed adaptable systems for classifying time series data that detect concept drift when the drop in accuracy across consecutive timestamps exceeds a decision threshold. Kalita et al. [105] detect drift by analyzing an alternative metric, namely the loss function used to train the classification model.

More elaborate methods for supervised drift detection are also adopted in the reviewed works, with adaptive windowing among the most widely used. ADWIN keeps a dynamic sliding window w containing the last n evaluations of the system's classification error. w is split into two adjacent subwindows w_1 and w_2, where the former represents older evaluations while the latter represents the most recent ones. When the absolute difference between the subwindow mean values exceeds a cutoff value ϵ, ADWIN infers that concept drift has occurred due to increased model error. In such a scenario, the older elements in w are removed until the mean values of w_1 and w_2 no longer differ significantly. Paim and Enembreck [19] and Pasikhan et al. [97] introduced evolving ensemble machine learning models for data classification, leveraging supervised ADWIN to detect concept drift and allow model updating.

Another frequent method for detecting drift using labeled data is the drift detection method [42]. DDM is based on the model's error probability at time t, denoted p_t, representing the likelihood of misclassification for the current sample under analysis. p_t is estimated as the ratio of incorrect inferences to the total number of processed samples n in previous timestamps. The method also approximates the dispersion of p_t as the standard deviation $\sigma_t = \sqrt{p_t(1-p_t)/n}$. The minimum observed values for the probability and its dispersion, p_{min} and σ_{min}, are also stored for future reference. DDM assumes that under a static scenario, the value of p_t should decrease through time as the model observes more samples. Significant increases in the probability p_t may signal the occurrence of concept drift. If $p_t + \sigma_t >= p_{min} + 2 \times \sigma_{min}$ then DDM raises a warning signal, indicating a probable concept drift start. The algorithm also begins storing data samples in its short-term memory for later model updates. Eventually, if $p_t + \sigma_t >= p_{min} + 3 \times \sigma_{min}$, then DDM confirms the drift event, updates the classification model with samples stored in memory, and resets all variables to restart the concept monitoring process. Nixon et al. [24], and Noori et al. [90] discussed concept drift-aware intrusion detection systems that adopt DDM to detect traffic changes and update the classification model.

One critical drawback of DDM is its difficulty in capturing slow, gradual drifts, as the drift threshold may take too long to trigger, causing short-term memory overflow [108]. The Early Drift Detection Method is proposed to detect these subtle drift events. Network administrators should choose the appropriate drift detection method based on the expected drift types (e.g., abrupt or sudden) that may arise in their target network environment, or implement a hybrid approach to identify multiple categories of drift if necessary. EDDM works similarly to DDM, but for each timestamp t, it keeps track of the average distance between two consecutive incorrect inferences, p'_t, and its respective standard deviation, σ'_t. The maximum observed distance p'_{max} and standard deviation σ'_{max} are also tracked. When p'_t and σ'_t begin to decrease over time, the classification model is committing mistakes more frequently, which may be the early evidence of an incoming concept drift event. Therefore, if $(p'_t + 2 \times \sigma'_t)/(p'_{max} + 2 \times \sigma'_{max}) < \alpha$ then EDDM raises the warning alarm, storing the newly incoming samples. When $(p'_t + 2 \times \sigma'_t)/(p'_{max} + 2 \times \sigma'_{max}) < \beta$, the drift level is reached, and the model is updated with stored samples while variables are reset. Anithakumari and Mishra [88] implemented an intrusion detection system adaptable to concept drift in non-stationary IoT networks. They conducted a comparative analysis between EDDM and alternative drift detection algorithms, namely Hoeffding drift detection method (HDDM) and ADWIN. Extensive

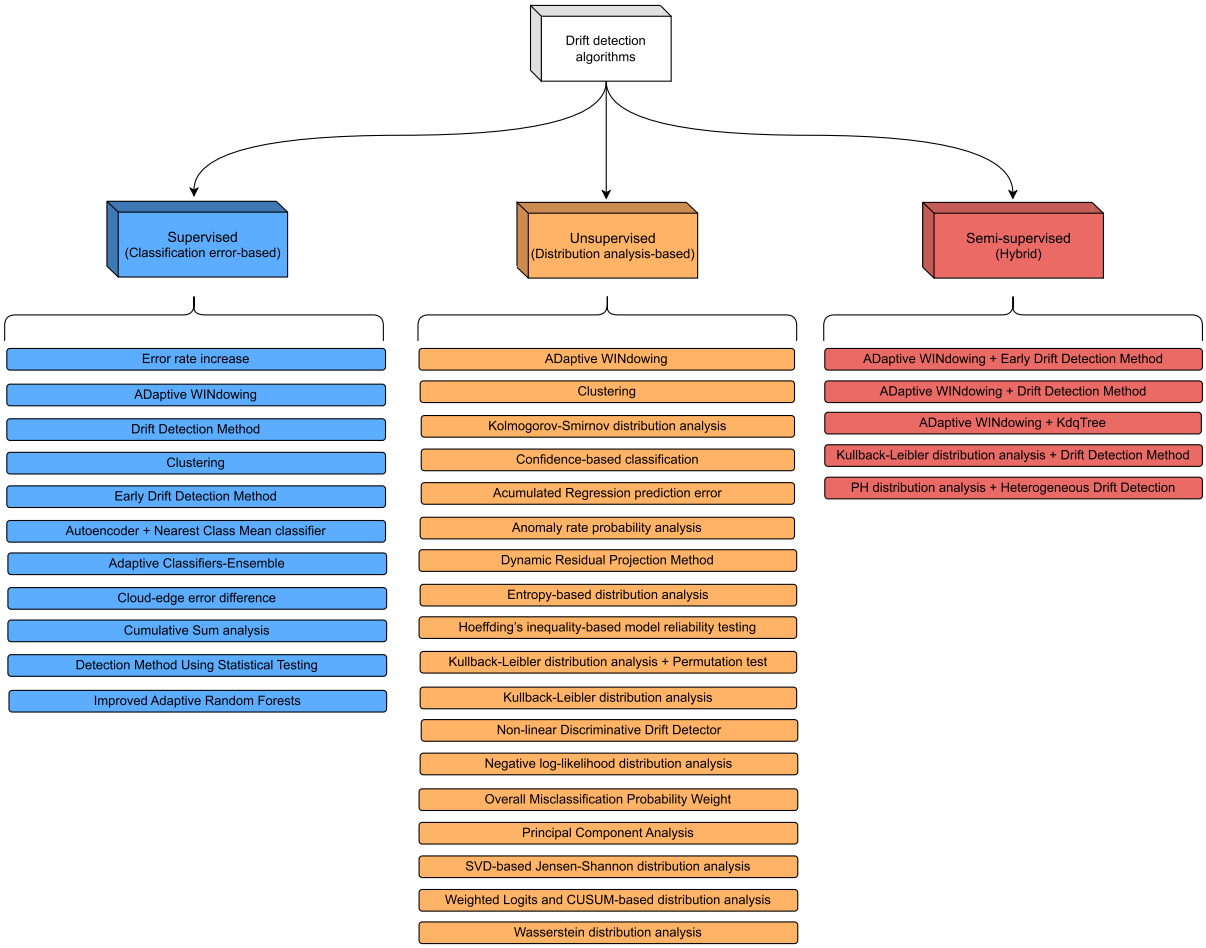


FIGURE 10. Drift detection algorithms taxonomy.

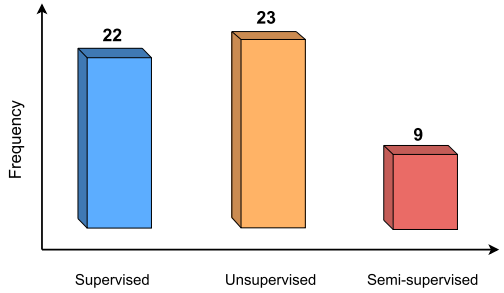


FIGURE 11. Drift detection paradigm frequency.

validation experiments across the IoTID20, EDGE-IIoT, Aalto, and RT-IoT22 benchmarks conclude that EDDM achieves the optimal solution.

2) UNSUPERVISED

The independence from labeled data for identifying drifting network behavior is the hallmark of an unsupervised algorithm. Such methods commonly compare statistics of past and incoming traffic to detect significant variation in numerical features. Beyond its applicability in supervised settings, the adaptive windowing method has also achieved remarkable performance in unsupervised ones. The unsupervised ADWIN maintains two adjacent sliding windows, w_1 and w_2, each containing raw features from the previous and currently observed network traffic, respectively. The absolute difference between the mean values of w_1 and w_2 is calculated. If it exceeds a threshold δ, ADWIN signals a distribution drift event. The drifting samples are stored for later updates to the learning model. Abdulboriy and Shin [76] described an incremental ensemble learning method for training a voting-based hybrid model aimed at network intrusion detection. They utilized unsupervised ADWIN to keep their system robust against network behavior drift. Singh et al. [73] developed an explainable deep-learning-based NIDS that is self-adaptable to non-stationary networks using the unsupervised ADWIN algorithm.

Table 6 reveals that most unsupervised drift detection methods are founded on the analysis of the statistical distribution of historical and recent samples. ADWIN performs such analysis using mean-value calculations, whereas other methods use more mathematically complex metrics. For example, Selvam and Balasubramanian [100] apply a machine learning model to produce probability estimates for the incoming IP flows in the network. They use the Kolmogorov-Smirnov test

TABLE 6. Drift detection algorithms. SP: Supervised, US: Unsupervised, SS: Semi-supervised.

#	Algorithm	Paradigm	Works
1	Error rate increase	SP	[20], [71], [72], [80], [85], [92], [99], [105]
2	ADaptive WINdowing	SP/US	[19], [26], [73], [74], [76], [97]
3	ADaptive WINdowing + Early Drift Detection Method	SS	[77], [79], [81], [104]
4	Drift Detection Method	SP	[24], [42], [89], [90]
5	Clustering	SP/US	[18], [62], [64]
6	Kolmogorov-Smirnov distribution analysis	US	[58], [60], [100]
7	ADaptive WINdowing + Drift Detection Method	SS	[65], [78]
8	Confidence-based classification	US	[45], [46]
9	Early Drift Detection Method	SP	[88], [89]
10	ADaptive WINdowing + KdqTree	SS	[107]
11	Autoencoder + Nearest Class Mean classifier	SP	[41]
12	Adaptive Classifiers-Ensemble	SP	[89]
13	Acumulated Regression prediction error	US	[93]
14	Anomaly rate probability analysis	US	[95]
15	Cloud-edge error difference	SP	[25]
16	Cumulative Sum analysis	SP	[91]
17	Dynamic Residual Projection Method	US	[22]
18	Detection Method Using Statistical Testing	SP	[89]
19	Entropy-based distribution analysis	US	[94]
20	Hoeffding's inequality-based model reliability testing	US	[101]
21	Improved Adaptive Random Forests	SP	[75]
22	Kullback-Leibler distribution analysis + Drift Detection Method	SS	[66]
23	Kullback-Leibler distribution analysis + Permutation test	US	[37]
24	Kullback-Leibler distribution analysis	US	[103]
25	Non-linear Discriminative Drift Detector	US	[68]
26	Negative log-likelihood distribution analysis	US	[87]
27	Overall Misclassification Probability Weight	US	[61]
28	PH distribution analysis + Heterogeneous Drift Detection	SS	[83]
29	Principal Component Analysis	US	[47]
30	SVD-based Jensen-Shannon distribution analysis	US	[106]
31	Weighted Logits Variation and CUSUM-based distribution analysis	US	[56]
32	Wasserstein distribution analysis	US	[23]

to assess the divergence between the cumulative distribution function (CDF) of these probabilities and the CDF of the training IP flows. Mrudula et al. [103] proposed a security system that detects network drift by evaluating the magnitude of the Kullback-Leibler Divergence between the feature distribution of past and latest traffic samples. The classifier proposed by Cao et al. [94] outputs a probability distribution for each analyzed network message, where a concentrated distribution represents that the model is highly confident in the sample's classification. An increase in the information entropy of the output distribution may be interpreted as concept drift, as prediction probabilities become more dispersed, indicating a rise in model uncertainty.

Some authors use strategies beyond distribution analysis for unsupervised drift detection. dos Santos et al. [45] and H. Wu et al. [46] proposed machine learning-based traffic classification systems that can adapt to dynamic environments. Their models classify traffic samples with a confidence probability; inferences with confidence values below a threshold are considered drifting samples. Schwengber et al. [68] utilize the non-linear discriminative drift detector (D3N) to improve their botnet detection system's resilience to drifting network behavior. D3N maintains a sliding window of older and newer samples, in which the older samples receive a different pseudo-label than the newer ones. If these sets of samples are separable by a decision tree classifier with an area under the ROC curve exceeding a threshold, then D3N assumes a drift event has occurred. Wang et al. [93] developed an LSTM regression model to predict future traffic and detect anomalies in Industrial IoT environments. Drift signals are triggered when the model's accumulated prediction error (e.g., mean squared error) surpasses a given threshold.

3) SEMI-SUPERVISED

Semi-supervised drift detection comprises hybrid algorithms that combine supervised and unsupervised methods. They utilize both labeled and unlabeled data to monitor the evolution of traffic behavior from multiple perspectives. The most frequent semi-supervised strategy in the reviewed works is to combine the unsupervised ADWIN with a classification-error-based method. Wu et al. [77] proposed an ensemble of online learning models for anomaly detection in IoT. Such models are integrated with ADWIN and EDDM for identifying abrupt and gradual concept drifts in the network. Jiangjiang Zhang et al. [79] discussed an IoT intrusion detection system based on a novel ensemble model. Their system detects concept drift if and only if the ADWIN and EDDM methods simultaneously trigger an alert for the traffic under analysis. Saqib et al. [65] introduced a drift-resilient IDS for volumetric attack identification in programmable networks. The system detects traffic evolution using DDM, which is subsequently double-checked with an

unsupervised ADWIN method to confirm or refute the occurrence of drift. Nonetheless, alternative strategies can also be used to implement a hybrid adaptive IDS. For instance, Jain et al. [66] described a network anomaly detection system that uses k-Means for feature extraction and SVM for data classification. Their system is robust to dynamic environments by detecting drift through a dual approach that combines Kullback-Leibler divergence analysis and the Drift Detection Method, and by retraining the learning models.

E. DRIFT ADAPTATION

In this section, we discuss the summary of the drift adaptation algorithms gathered from the selected works, answering the fourth research question (RQ4). Fig. 12 presents a taxonomy of these methods, organized by their primary target-update component for evolving environments. Model-centric strategies involve adapting non-ensemble machine learning models through retraining or incremental/online learning [41], [66]. Ensemble-based methods comprise adjusting the structure or the weighted decision voting parameters of committee models [84], [97]. Data & representation regards feature and instance space management, through stable feature extraction and dynamic reselection, as well as knowledge pool updating [67], [81]. Finally, meta-strategies & external factors encompasses techniques ranging from human intervention to hyperparameter tweaking [105], [106].

Fig. 13 illustrates the frequency of articles within the drift adaptation algorithm taxonomy. Model-centric approaches predominate among the reviewed works, accounting for approximately 58% of the total (40 out of 69). The remaining distribution is as follows: ensemble-based methods (14 papers, 20%), data & representation (12, 18%), and meta-strategies & external factors (3, 4%). Table 7 catalogs 27 specific algorithms underlying these macroscopic categories, correlating each strategy to the articles that implemented them. Triggered retraining was the most common algorithm, retraining the target machine learning model when an external detector signals drift. The second most common procedure is incremental learning, which encompasses the incremental addition of knowledge to a model. Periodic retraining appears in 5 works and concerns solutions that assume drift as a given and periodically retrain their models with newer data. These and other solutions, such as online learning, active learning, ensemble weighted voting update, dynamic ensemble selection, and feature reselection, were combined within distinct learning paradigms to sustain the model's performance and address non-stationary data.

Adaptation techniques are often directly tied to the target ML model's learning strategy; thus, the most common adaptation approaches span more than one of the considered training paradigms, i.e., supervised, semi-supervised, and unsupervised, as shown in Table 7. Supervised drift adaptation was employed in 46 works (67%), while unsupervised appeared in 12 (17%), and semi-supervised in 11 (16%). Even though most works rely on ground-truth, the latter are scarce in real-world scenarios because labeling data is time-, cost-, and labor-intensive. Some works aim to mitigate the unavailability and delay of labeling by employing various strategies. For instance, García and DeCastro-García [48] rely on conformal prediction to generate pseudo-labels for new unlabeled data. If ground-truth becomes available, even if delayed, it is also integrated into the online learning update process. Although, the article highlights that these pseudo-labels show inconsistent performance in cybersecurity datasets. Huang and Ma [58] propose a semi-supervised intrusion detection system that mitigates the dependence on high-quality, immediately labeled data. The system incorporates incremental k-means to separate normal and abnormal traffic, and the density-based stream model to further identify these anomalous samples. As a new attack is identified, it is temporarily classified as unknown until a human agent manually labels it.

Besides the unavailability of labeled data, a challenge that often appears in dynamic systems is catastrophic forgetting. This phenomenon, also known as catastrophic interference, refers to the degradation of an ML model's previously acquired knowledge as sequential incorporation of new data drastically updates the underlying model's parameters [21], [23], [69]. Some of the surveyed works address this issue explicitly. For instance, Saravanan and Balasubramanian [80] consider both new and historical data when retraining their system. This technique, called buffer replay, reassures older patterns by keeping them in the training pool. However, storing a continuous flow of data is resource-intensive. Thus, Liu et al. [56] implement an undersample strategy to store only the most representative portion of data for buffer replay. Other strategies such as data preprocessing, preservation of older models, and layer regularization are also explored [69], [82], [107].

1) MODEL-CENTRIC ADAPTATION

The drift adaptation algorithms are categorized as model-centric when their evolution strategy involves directly modifying the target ML model's learnable parameters or architecture. We chose to dissociate ensemble-based systems from this category because their structure fundamentally depends on multiple base learners, thereby yielding specific adaptation solutions. Fig. 12 displays this taxonomy excerpt subdivided into two approaches: retraining and incremental & online learning.

Retraining algorithms aim to update natively stationary models by training a similar model from scratch, using new data, but not necessarily limited to it. The most commonly employed drift adaptation solution was triggered retraining, which appeared as the only strategy in 12 analyzed papers, as shown in Table 7. This algorithm retrains the target model once a drift is identified by an external detector. Conversely, periodic retraining appeared singly in 4 research papers, assuming drift as a certain and training models from scratch at fixed intervals.

The other three methods based on retraining appeared in the surveyed literature as hybrid and iterative solutions.

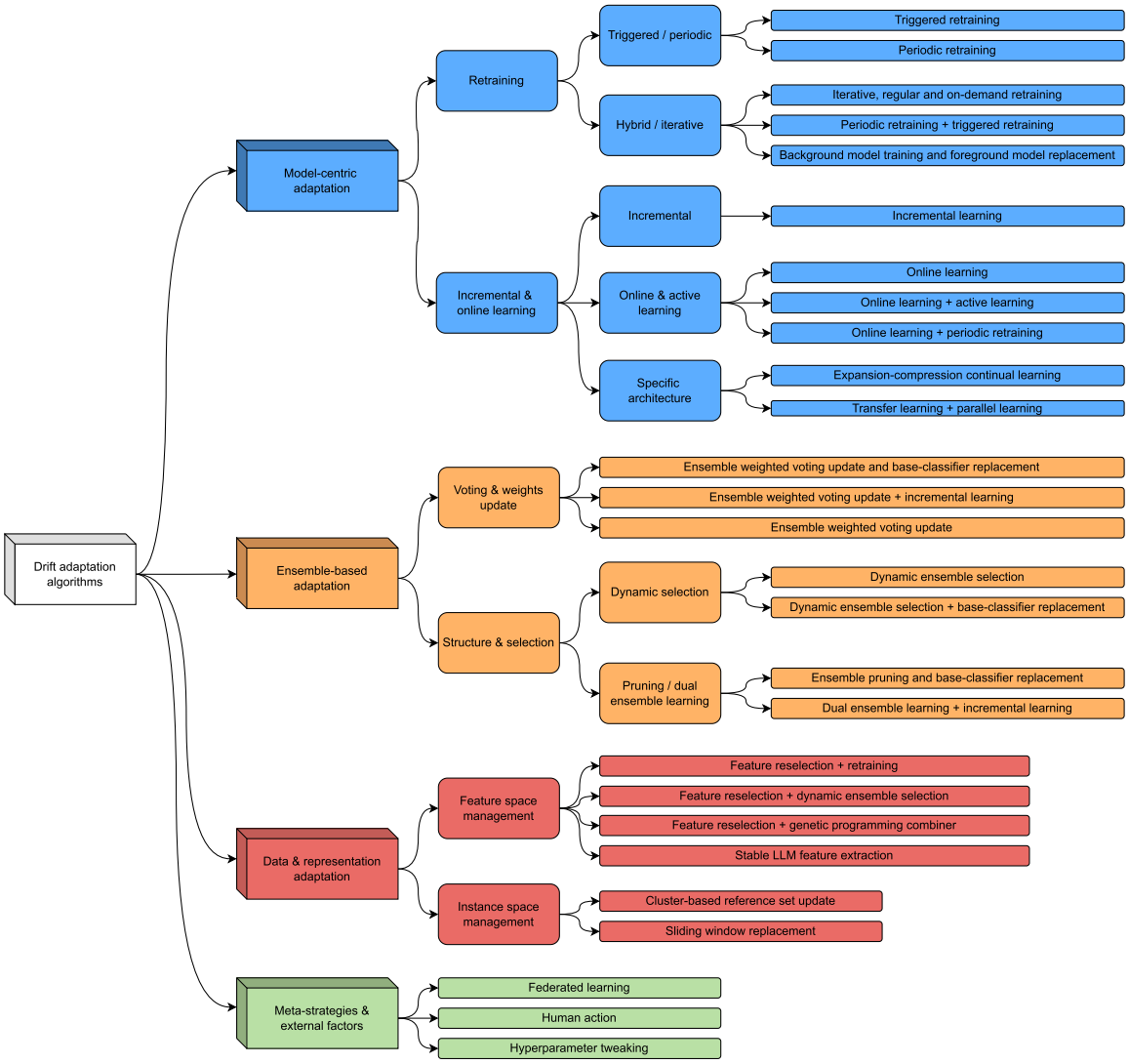


FIGURE 12. Drift adaptation algorithms taxonomy.

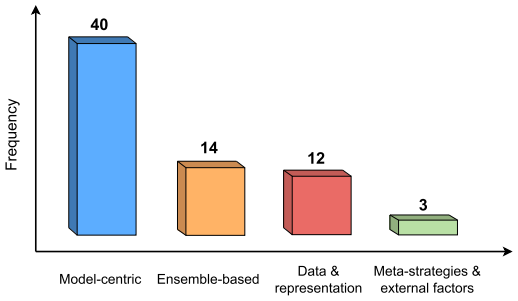


FIGURE 13. Drift adaptation taxonomic frequency.

Nixon et al. [24] proposed training a background autoencoder when drift is alerted by a DDM algorithm. When the drift is confirmed, the foreground model is replaced. Wang et al. [93] compared three retraining strategies: iterative, regular, and on-demand. Iterative offline updates retrain the model on every batch of new data. Regular updates fall under the periodic retrain umbrella, while on-demand updates recreate a model from scratch when the accumulated error exceeds a threshold. The last two approaches were identified by the authors as the most effective for maintaining the model up to date, whilst the on-demand update presented lower accumulated error and lower resource usage. Finally, Zhang et al. [25] implemented a hybrid procedure combining periodic and triggered retraining. Their system relies on a large model in the cloud that serves as an oracle for ground truth and is assumed to be immune to drift. Multiple tiny models are placed on the edge to perform anomaly detection. As edge models drift or a predetermined period of time passes, they are retrained.

Incremental & online Learning algorithms promote continual aggregation of new data on a model's knowledge. In this context, works often employ terminologies such as continual, lifelong, incremental, online, stream, and active learning in

TABLE 7. Drift adaptation algorithms. SP: Supervised, US: Unsupervised, SS: Semi-supervised.

#	Algorithm	Paradigm	Works
1	Triggered retraining	US/SS/SP	[18], [22], [37], [46], [60], [65], [66], [80], [89], [94], [95], [100]
2	Incremental learning	US/SS/SP	[21], [23], [26], [41], [56], [72], [73], [74], [96], [101], [107]
3	Ensemble weighted voting update and base-classifier replacement	SS/SP	[61], [84], [87], [91]
4	Online learning	US/SP	[47], [48], [75], [85]
5	Periodic retraining	US/SS/SP	[45], [49], [63], [98]
6	Cluster-based reference set update	US/SS	[57], [64], [67]
7	Ensemble weighted voting update + incremental learning	SP	[76], [77], [78]
8	Feature reselection + retraining	SP	[59], [81], [104]
9	Online learning + active learning	SS/SP	[62], [82], [103]
10	Base-classifier replacement	SP	[20], [97]
11	Dynamic ensemble selection	SP	[42], [88]
12	Feature reselection + dynamic ensemble selection	SP	[79], [83]
13	Feature reselection + genetic programming combiner	SP	[86], [90]
14	Background model training and foreground model replacement	SP	[24]
15	Dual ensemble learning + incremental learning	SS	[58]
16	Dynamic ensemble selection + base-classifier replacement	SP	[19]
17	Ensemble weighted voting update	SP	[92]
18	Expansion-compression incremental learning	SP	[69]
19	Federated learning	SP	[70]
20	Human action	SP	[106]
21	Hyperparameter tweaking	SP	[105]
22	Iterative, regular and on–demand retraining	US	[93]
23	Online learning + triggered retraining	SP	[99]
24	Periodic retraining + triggered retraining	SS	[25]
25	Sliding window replacement	US	[68]
26	Stable LLM feature extraction	SS	[102]
27	Transfer learning + parallel learning	SP	[71]

this context. Among these, active learning designates systems that rely on the interactive participation of a human oracle for data labeling [62], [82]. The remaining methods focus on data ingestion and overlap significantly. Incremental learning is the broadest category, describing models that acquire knowledge additively over time, typically by processing accumulated batches of new data [21], [72]. Online and stream learning refine this concept by updating the model one instance at a time [47]; stream further emphasizes the uninterrupted, real-time nature of the data flow [24], [89]. Finally, continual and lifelong learning explicitly addresses the system's stability, prioritizing the integration of new data while preserving previously acquired knowledge [23], [69]. Since we address catastrophic forgetting in a dedicated section, this survey treats continual and lifelong as either incremental or online, depending on the granularity of updates. The distinction between stream and online learning is also omitted.

Adaptation solutions that rely on incremental learning as the main tool for non-stationarity accounted for 11 collected works. Among these, tree [26], [74], [96], [107] and autoencoder-based [23], [41], [101] systems where the most common. Online learning was implemented alone in 4 works, combined with active learning in 3, and with triggered retraining in 1. The integration of active learning provides ground-truth elucidation in semi-supervised approaches, as in Liu et al. [82], where a human specialist manually labels uncertain instances. In supervised solutions, the oracle categorized new drifted samples [62], [103]. Triggered retraining was considered by Lu et al. [99] to offline retrain the model from scratch when online performance degrades.

It is worth noting that two other works added knowledge to their systems using specific architectures that natively support adaptation. Soltani et al. [69] presented a distributed intrusion detection system that incorporates an expansion-compression incremental learning method. Their solution increases the number of neurons in the dense layers of the distributed models when new data is received. The central model leverages knowledge distillation to incorporate the agents' learned parameters. Once the main model is updated, the agents replace their local neural networks. Eskandari and Khotanlou [71] implemented a combination of transfer and parallel learning. An RNN-based model is trained and constantly fine-tuned (i.e., incrementally updated) on new data. A parallel replica is retrained from scratch for every batch of new data. When a gradual drift occurs, and the base model's updated performance is lower than the parallel's, the main model is replaced. For abrupt drift, the performance comparison is ignored, and the parallel model is promptly implemented.

2) ENSEMBLE-BASED ADAPTATION

Ensemble-based adaptation algorithms comprise committee-specific adjustments, such as updating the weighted voting parameters and modifying the ensemble structure. Thus, the taxonomy for ensemble-based algorithms is further split into voting & weights update and structure & selection, as shown in Fig. 12. The first subcategory includes 7 works across 3 distinct approaches. The most straightforward strategy is to update the base learners' weights in the ensemble's decision voting, as implemented by Abbasi et al. [92]. Their method is prone to accuracy loss in dynamic environments because the ensemble's knowledge is static. To mitigate this limitation,

some works replace their worst-performing classifiers with newly trained ones, while adjusting their decisive weights in the ensemble [61], [84], [91]. Since retraining a model from scratch may be cost-intensive, incremental learning was also combined with updating voting weights [76], [77], [78].

Structure & selection ensemble adaptation comprises techniques that modify the structure of the ensemble. Algorithms that dynamically select coalitions of base learners according to their classification performance on current data accounted for 3 papers. The primary technique, named dynamic ensemble selection, works by maintaining a pool of base learners and dynamically composing an ensemble based solely on the decision of the best-performing subgroup of classifiers [42], [88]. Dynamic selection alone does not update the model's knowledge; rather, it temporarily relies on the best-performing static ones. To integrate new data into this paradigm, Paim and Enembreck [19] used an external drift detector for each base classifier, replacing it with a newly trained model when its performance declined.

Alternatively, to avoid the resource overhead of storing and selecting multiple unused models, structural strategies can be adopted. The most common structural tweaking analyzed was base-classifier replacement. This procedure comprehends substituting obsolete base learners when a drift is detected [20], [97]. To circumvent the delay of training a new model when drift is detected, Huang and Ma [58] proposed a dual ensemble learning system that leverages coextensive active-standby ensembles. While one model classifies new data, the other is trained incrementally. The active committee is replaced when drift is detected.

3) DATA & REPRESENTATION ADAPTATION

Techniques based on feature or instance space management are categorized as data & representation adaptation. Within this category, feature space management primarily addresses feature drift. This is achieved either by dynamically reselecting features as they become more significant or by assuming a stable representation immune to shifting distributions. Feature reselection is frequently combined with strategies such as dynamic ensemble selection, genetic programming combiners, and triggered/periodic retraining, as listed on Table 7. For instance, Malathy et al. [83] utilized Select K-Best and Poisson Inclusion to reselect representative features while dynamically choosing the top 2 out of 4 base classifiers.

Regarding genetic programming, works such as Noori et al. [90] and Shyaa et al. [86] used it to generate graphs that combine base learner decisions. On the other hand, Yang et al. [81] employed the Pearson correlation coefficient to select the k most correlated features on drifted data and trigger model retraining when drift occurs. Rabash et al. [59] periodically retrained their system by genetically determining the updated most important feature set. Conversely, assuming stability in graph abstraction of network representations, Lin et al. [102] proposed a static and drift-immune feature selection method, in which a large language model (LLM) was trained using a technique called BURST graph matching.

Instance space management refers to the updating of reference data samples in both cluster- and sliding-window-based systems. Cluster-based reference set update employs techniques that dynamically incorporate new data points, thereby creating additional clusters. Scaranti et al. [67] model data through potential and core micro-clusters. When a new pattern is detected, a potential micro-cluster is created to accommodate it. Recurring potential clusters are promoted to core clusters, representing a new consolidated standard. Gudapati et al. [64], and Liao et al. [57] are also based on micro-clusters, although both solutions attribute confidence weights to their clusters. The confidence of a cluster is automatically updated regarding its ability to model new data. The work of Schwengber et al. [68] trained their intrusion detection system in a pseudo-labeled sliding window of the current network concept. The content of this window is dynamically regulated, whereby the system discards the entire block of older instances upon detecting a drift to adapt to new statistical distributions. If the concept remains unchanged, only a small portion of the window is replaced by newer data.

4) META-STRATEGIES & EXTERNAL FACTORS

Miscellaneous algorithms were organized into meta-strategies & external factors, as they depend on federated learning, human action, or hyperparameter tweaking. Quin et al. [70] implemented a distributed network traffic classification system that operates via communication rounds between edge nodes and the server. In each new round, the central model incorporates node expertise via Federated Averaging and distributes the updated classifier to the edges. These nodes update the received model locally using a joint objective function that combines supervised and contrastive losses.

Regarding human action, the system proposed by L. Wang et al. [106] depends on manual failure analysis and execution of adaptive mitigation mechanisms when drift is detected. The employed operations comprehend service offloading and traffic interception, to restore system stability. Lastly, even though some works use hyperparameter tweaking as part of their adaptation solution, Kalita et al. [105] rely solely on it. Their Support Vector Machine-based intrusion detection system is tailored to achieve dynamicity by tuning its hyperparameters. Three approaches are explored depending on the gravity of the drift: a complete search for parameters using moth-flame optimization (MFO) when a major drift is detected; a lightweight MFO initialized with prior explored parameters for minor drifts exceeding a threshold; and a simple search within the stored prior knowledge list is conducted for the smallest variations.

5) CATASTROPHIC FORGETTING

As dynamic models continuously aggregate new knowledge, old learned patterns tend to be forgotten or degraded, whether

TABLE 8. Catastrophic forgetting mitigation strategies.

#	Algorithm	Works
1	Buffer Replay	[20], [23], [46], [63], [65], [66], [80], [93]
2	Buffer Replay with undersampled historic data	[37], [56], [99], [100]
3	Data preprocessing	[41], [82], [87]
4	Generative Buffer Replay	[18], [71]
5	Obsolete models preservation	[42], [107]
6	Bias correction	[21]
7	Elastic Weight Consolidation	[101]
8	Elastic Weight Consolidation + layer freezing	[91]
9	Regularization + Buffer Replay	[69]
10	Transfer learning	[45]
11	Transfer learning + Feature Count Maximization Principle	[86]

due to neurons' weights being overwritten in neural networks or because of obsolete base classifiers being replaced in ensembles, for example. The trade-off between the ability to learn new tasks and retain old knowledge poses an essential problem in adaptive systems, known as the stability-plasticity dilemma [109]. While some works favor plasticity over stability justified on the evolving nature of data [47], [48], [88], real-world applications for changing environments should preserve the validity of historical expertise, which may become active in the future [23], [37]. Thus, many proactive strategies emerge to mitigate catastrophic forgetting, also known as catastrophic interference.

Catastrophic forgetting was addressed in 25 of the 69 collected research papers, displayed in Table 8. Two works, specifically Lin et al. [102] and Junjie Zhang et al. [25], proposed the use of a stable and stationary large model. Since their approach is stationary, no strategy was explored to mitigate forgetting. Similarly, Abbasi et al. [92] only update the decisive weights of base classifiers, thereby preserving the overall ensemble's expertise.

The most common procedure is Buffer Replay, present singly in 8 works and with alternative implementations in other 7. This strategy comprises the conservation of historic data in the update set, along with new samples [66], [93]. Some replay implementations aim to mitigate labeling delay and unavailability by considering pseudo-labeled drifted data [63], [65]. Since storing data indefinitely becomes unfeasible in the long term, four works kept representative undersampled sets of historic data to reduce memory overhead [37], [56], [99], [100]. Another replay alternative is generative replay, which appeared in 2 works [18], [71]. This variation trains a neural network capable of recreating historical instances while avoiding excessive storage resource consumption. Traditional replay was also combined with layer-wise regularization in the study by Soltani et al. [69] to constrain gradients and prevent overwriting neuron weights.

Seven algorithms accounted for the remaining 10 studies, where data preprocessing and obsolete models preservation appeared in more than one study. Data preprocessing refers to assigning weights directly to data, as proposed by Liu et al. [82], or to preserving older data clusters, as in Kuppa and Le-Khac [41], and Li et al. [87]. Conversely, obsolete models were retained within the ensembles, as this method focused solely on integrating new models [42], [107].

Elastic Weight Consolidation was leveraged to calculate parameter importance, updating weights via regularization [101] and, in a specific instance, through layer freezing [91]. Regarding transfer learning, two approaches were identified: dos Santos et al. [45] applied it to distributed systems. In contrast, Shyaa et al. [86] utilized the feature count maximization principle to retain the weights of inactive features, ensuring they are not relegated to oblivion. Finally, to address the bias toward new classes, Cerasuolo et al. [21] proposed a bias correction technique that modifies the model's architecture by adding a parameterized layer.

F. METRICS

In this section, the metrics of the analyzed works are organized by category, answering the fifth research question (RQ5). Metrics are fundamental to evaluating the performance aspects of each proposed work. The chosen metrics must align with the work's goals to properly measure the success of the proposed design. Not every article makes concept drift detection and mitigation its primary goal; thus, most metrics encountered measure the proposed model's performance in detecting and mitigating anomalies rather than in handling concept drift. Internal metrics, typically used for model selection or optimization, fall outside the scope of this survey. We focus exclusively on external metrics, which evaluate the model's generalization performance on unseen data, independent of the training set. Fig. 14 illustrates a taxonomy of the considered performance metrics, separated by evaluation scope.

1) CONCEPT DRIFT HANDLING

Only 7 of 69 works presented metrics for uniquely measuring concept drift handling. Traditional metrics such as precision, recall, and F1-score are the most frequent. However, whilst they are suitable for anomaly detection in both dynamic and stationary scenarios, there are other ways to showcase drift detection techniques specifically. All works that used

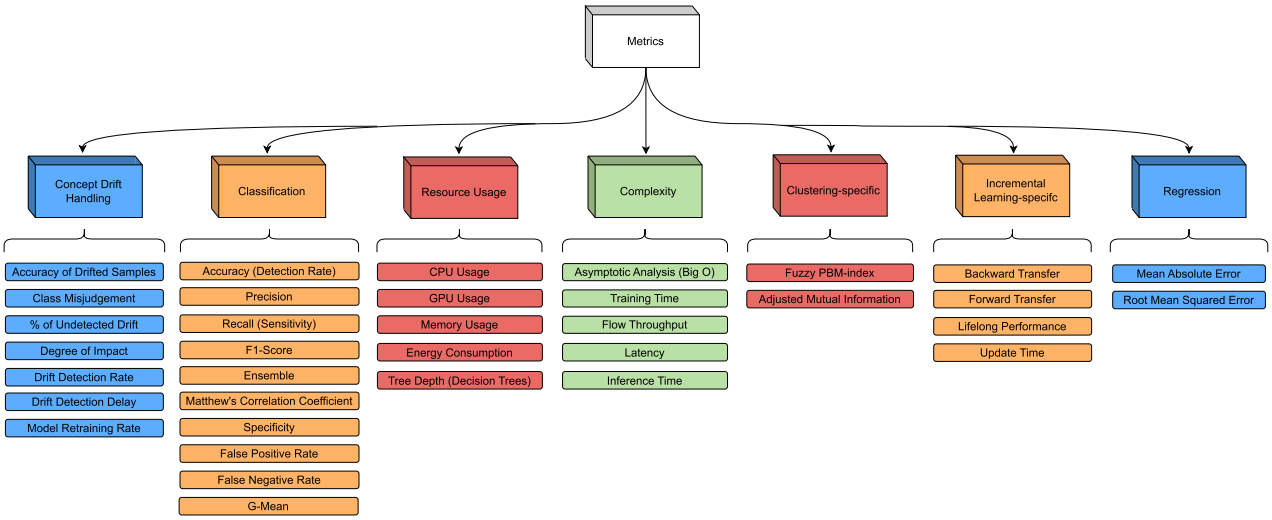


FIGURE 14. Performance metrics taxonomy.

drift-specific metrics are listed, given their scarcity and singular appearance.

Wu et al. [46] calculate the Accuracy of Drifted Samples dividing the correctly detected by the total drifted traffic. This approach is similar to the Class Misjudgement metric, used by Liao et al. [57], in which the authors calculate the rate of misjudgement of existing classes as a drifting one. They also present the percentage of drifting elements undetected by the system (% of Undetected Drift). Those three metrics are a straightforward way of quantifying the system's detection performance, but require the developers to know the ground-truth labels not only in terms of maliciousness but also whether the data are drifting or simply outliers.

Another measurement for drift mitigation performance is the Degree of Impact as presented by Zhang et al. [25]. This metric represents the difference in accuracy between a specialist, up-to-date model and an older one. Similarly, Zhang et al. [79] also measure the Drift Detection Rate, by calculating the precision, recall, and F1-score on drift detection, relying on CD annotations. Similar to previous metrics, these depend on specialized information that can be laborious to obtain in real-world environments.

Since the intrusion detection problem is particularly time-sensitive, the delay in detecting concept drift is key. The Drift Detection Delay may be measured both in time and in the number of instances until detection, as employed by Qiao et al. [22], and by M. Liu et al. [56], respectively. On the other hand, Saqib et al. [65] measure the Model Retraining Rate over time, aiming for its minimization to avoid performance issues, traffic disruptions, and packet loss without interfering with other detection metrics.

2) CLASSIFICATION

Classification-based measures represent the most prevalent category of metrics in the surveyed papers. They rely mostly on quantifying true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN), which are often organized in a confusion matrix. These measurements enlighten authors about the performance of their systems and help them make decisions based on the preferred error trade-off for their application scenario. Thus, the selection of specific metrics often reflects the unique constraints of the proposed model, such as false alarm minimization or class imbalance addressing.

Accuracy, Precision, Recall, and F1-Score are the most commonly used metrics, being defined by functions (1) to (4). They are often referred to by different names, such as detection rate for accuracy and sensitivity (or true positive rate) for recall. Each one of them represent different forms of correctness: Accuracy represents how often the classification is correct among all samples; Precision is the rate at which positive answers are correct; Recall shows the classifier's capability of identifying all the positive elements; and F1-Score is the harmonic mean of Precision and Recall.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{1}$$

$$\text{Precision} = \frac{TP}{TP + FP} \tag{2}$$

$$\text{Recall} = \frac{TP}{TP + FN} \tag{3}$$

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{4}$$

The F1-Score is preferable to Accuracy for measuring performance on imbalanced datasets, where negative samples are more common than positive ones. However, it may return a biased value if the number of negatives is lower than the number of positives. In those cases, a more suitable metric is Matthew's Correlation Coefficient (MCC). Represented in (5), this metric considers every primitive metric of the confusion matrix, and, unlike F1-score, its value is not biased in case the positive and negative classes have unmatched

proportions.

$$MCC = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \tag{5}$$

In hypothesis testing, a Type 1 error (false positive) occurs when a null hypothesis is incorrectly rejected due to statistical noise or misleading data. Conversely, a Type 2 error (false negative) occurs when a false null hypothesis is incorrectly accepted, failing to detect a discrepancy that more precise measurements would have. In the context of network intrusion detection, Type 1 errors represent the misclassification of legitimate traffic as malicious, while Type 2 errors correspond to missed attacks. To quantify the trade-off between these error types, specialized metrics become essential. Specificity, or true negative rate (TNR), quantifies the model's ability to classify negative samples (normal traffic). Closely related is the false positive rate (FPR), which measures how often a benign sample is incorrectly flagged as anomalous. False negative rate (FNR) measures the rate of attacks the system fails to detect and is equivalent to the complement of the recall. The functions for these metrics are represented on (6), (7), and (8), respectively.

$$\text{Specificity} = \frac{TN}{TN + FP} \tag{6}$$

$$FPR = \frac{FP}{FP + TN} = 1 - \text{Specificity} \tag{7}$$

$$FNR = \frac{FN}{FN + TP} = 1 - \text{Recall} \tag{8}$$

Finally, the G-Mean is the geometric mean of arbitrary class-wise performance measures, usually combining Specificity and Recall. Its primary purpose is to mitigate bias toward majority classes, providing an indicator of performance deficiencies even when the overall accuracy remains high due to class imbalance. The G-Mean formulation is presented in (9), where N represents the number of selected metrics and m_i denotes the value of the ith component metric.

$$\text{G-Mean} = \Big(\prod_{i=1}^{N} m_i\Big)^{\frac{1}{N}} \tag{9}$$

3) RESOURCE USAGE

Network anomaly detection systems are designed to receive network traffic and update forwarding rules continuously. Thus, measuring resource usage is necessary to evaluate the system's cost relative to hardware utilization. Among the surveyed literature, works presented metrics such as CPU and GPU Usage, Memory Usage, and Energy Consumption [25], [42], [61], [71].

Specific metrics may also be employed in systems to handle specific cases, such as average Tree Depth in the ensemble of Hoeffding trees [19]. This case applies to both resource usage and algorithmic complexity, as the ensemble and tree sizes grow during training.

4) COMPLEXITY

Algorithm complexity is a way to estimate the cost and scalability of a proposed system relative to the cost of processing input samples. Pasikhani et al. [42] use Big O notation to address the entire system's complexity. Others measure indicators to it, such as Training Time, Flow Throughput, Latency, and Inference Time [58], [77], [81], [83]. These methods can be considered suitable for the network anomaly detection task, since they measure computing cost, and NIDS often operate in time-sensitive environments.

5) CLUSTERING-SPECIFIC METRICS

In the selected works, clustering methods were a frequent option for detecting classes and adapting the model to unseen data. Works such as Zhao et al. [96] and Wu et al. [46] applied, respectively, clustering accuracy and micro F1-score to evaluate the system's quality. However, clustering-specific metrics can be employed to measure performance in a more tailored way.

Some clustering systems may work with a variable number of groups created on demand by their algorithm. There is a sweet spot to the number of groups created, as too many groups become less meaningful to the categorization of data, and too few groups will not separate classes enough. Thus, it is important to judge the quality of those clusters and assign a score to them. The Fuzzy PBM-index used by Li et al. [87] is represented in (10) where K is the number of clusters, E is the deviation of the dataset, D is the maximal distance between the centers of the clusters, and J is a measure of the intrinsic variance within clusters. This metric rewards fewer groups, a greater distance between clusters, and how close the dataset points are to the centers of their own clusters. This provides a quantitative way to compare not only clustering algorithms but also how the same algorithm behaves with different numbers of clusters, enabling better decision-making when adding or removing clusters. This metric does not require labeling since it only measures the "geometry" of the clusters.

$$I_{Fuzzy-PBM} = \left(\frac{1}{K} \times \frac{E}{J} \times D\right)^2 \tag{10}$$

Another metric found in the reviewed works is the Adjusted Mutual Information (AMI) [96]. As opposed to PBM, this metric compares the clustering results to the dataset labels and can also be used to compare two sets of clusters. It is similar to an accuracy score, but it is capable of handling varying numbers of clusters for each class. This way, if a model separates a single class into two clusters but keeps the clusters "pure", it still receives a positive score. The metric is called "adjusted" because it addresses the standard mutual information (MI) flaw: it rewards models that create multiple small clusters, artificially inflating their score. The adjusted version subtracts the MI score that may occur by random chance, which could be achieved by creating multiple clusters. Its formula is represented in (11), where U and V are sets of clusters to be compared, such as the predicted labels

and the ground-truth annotations; MI and $E[MI]$ denote the observed and expected mutual information, respectively, and $H(\cdot)$ signifies the entropy of the partitions.

$$AMI(U, V) = \frac{MI(U, V) - E[MI(U, V)]}{avg(H(U), H(V)) - E[MI(U, V)]} \quad (11)$$

6) INCREMENTAL LEARNING-SPECIFIC METRICS

Incremental learning models are designed to be frequently, sometimes constantly, updated. They offer a dynamic framework that does not rely on historical datasets, which become obsolete over time. Instead, incremental learning methods process network traffic as new data arrives, either in batches or singular instances. To evaluate them, pure, traditional metrics such as accuracy do not work, as they treat the whole dataset statically, as a single set, without considering timing context.

Incremental learning metrics evaluate the model's adaptability and knowledge retention across training iterations by tracking standard metrics' evolution across sequential tasks. Therefore, metrics such as Backward Transfer (BWT) represent how new learned concepts affect previously encountered samples. Its function is described in (12), where T is the total number of tasks; $R_{i,i}$ is the model's performance based on any standard metric on task i immediately after it was trained on it; and $R_{T,i}$ is the performance after the model was trained on all tasks. In network anomaly detection, data can be fed incrementally to the model at different granularities, either in batches or as single instances. This metric shows the model's average performance after each training feed-forward cycle, thereby better reflecting how new data affects older data. A positive BWT indicates that, on average, newer data helps with older tasks, and a negative the opposite. Overall, it indicates the model's ability to resist catastrophic forgetting.

In contrast, Forward Transfer (FWT) assesses the model's ability to generalize old knowledge to future, unlearned concepts. As shown in (13), $R_{i-1,i}$ represents the performance of the model on batch i before it was trained on it, and b_i is the performance of a model that was not trained at all. This metric indicates how previous knowledge helps with new data compared to a completely untrained model. A positive FWT demonstrates that the system can generalize knowledge to new information, whereas a negative FWT means the opposite.

$$BWT = \frac{1}{T-1}\sum_{i=1}^{T-1}(R_{T,i} - R_{i,i}) \quad (12)$$

$$FWT = \frac{1}{T-1}\sum_{i=2}^{T}(R_{i-1,i} - b_i) \quad (13)$$

Lifelong Performance is a summary statistic that captures the cumulative performance across the model's entire operational history. It is similar to a raw performance measurement, but without considering future data collection. Backward Transfer, Forward Transfer, and Lifelong Performance metrics were employed in Faber et al. [23]. The standard performance measurement chosen as the base metric was receiver operating characteristic - area under the curve (ROC-AUC), used for binary classification that evaluates how well a model distinguishes between classes across all possible decision thresholds. The ROC curve visualizes the trade-off between true positive rate (sensitivity) and false positive rate as the threshold varies. The AUC transforms this performance history into a single number, where 1 represents a perfect system, and 0.5 or lower represents random guessing or worse.

Finally, Update Time measures the latency of integrating new data into the model, considering it operates in a real-time environment [99]. The lower the latency, the more likely the model is to operate at full capacity on newer data.

7) REGRESSION-SPECIFIC METRICS

In regression algorithms, the base for measuring performance is the inference error. The only two error metrics found were Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE), both from Wang et al. [93]. Their formulas are represented in (14) and (15) respectively, where y_i is the ith observation, $\hat{y}_i$ is the correct value for that element, and N is the total number of elements.

$$MAE = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i| \quad (14)$$

$$RMSE = \sqrt{\frac{\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}{N}} \quad (15)$$

MAE grows linearly with its inputs. It is better suited to avoid getting the result distorted by outliers. RMSE gives more weight to bigger values than smaller ones. Thus, it is more appropriate when the objective is to improve the model's consistency and generalization, rather than to perfect some samples while neglecting others.

VI. OPEN ISSUES AND FUTURE DIRECTIONS

In this section, we answer the final research question (RQ6). Relevant open issues identified during the study are discussed, indicating practical implementation challenges and future research directions. Drift adaptation is handled in the surveyed works through 27 gathered algorithms, revealing a tendency towards supervised, model-centric approaches, with retraining and incremental learning variants being the most common. Drift detection was explicitly addressed by the majority of research, yielding 32 algorithms, mostly unsupervised, and based on data distribution analysis. Furthermore, intrusion detection relied for the most part on shallow learning and static evaluation metrics. This trend can be related to the scarcity of representative computer network traffic datasets exhibiting drift.

A. DATASETS

Training machine learning models is directly hinged on data. As datasets represent real-world computer networks

more faithfully, it is expected that systems become more proficient at detecting attacks and intrusions. The lack of concept drift portrayal in currently available datasets poses as one of the main limitations to the applicability of IDS outside the laboratory. This limitation is compounded by privacy regulations (such as GDPR or LGPD), which restrict the collection of granular network traffic, thereby hindering the use of raw data for research and preventing the capture of authentic drift patterns that occur in production environments.

Even though 50 network traffic datasets were collected, only 4 effectively contained drift, namely the MAWI Working Group archive, DS2OS Traffic Traces, IP Maliciousness, and UGR'16. Only 5 works considered these datasets in their drift experimental benchmarking, leading the remainder of research to rely on data from other domains, to simulate non-stationarity artificially, or even disregard explicit empirical tests of drift resilience. Four main simulation techniques were outlined in our review, comprising class imbalance, data fusion, synthetic generation and injection, and geometric partitioning. Every method has a particular flaw, failing to represent precise feature interactions, network evolution, realistic traffic semantics, or temporal dependencies.

We highlight that researchers should further make use of available network traffic datasets that contain drift, rather than simulating it or overlooking drift evaluation altogether. To mitigate data scarcity, we encourage future research to focus on collecting or emulating computer network data that natively encompasses dynamicity.

B. NETWORK INTRUSION DETECTION SYSTEMS

Network intrusion and attack detection is a task that often leverages machine learning's ability to model intricate patterns in data. We identified a predominance in shallow, supervised learning strategies. Among these, the majority of works implement ensemble-based systems, especially variants of random forests such as the adaptive random forest. We observed a decline in the adoption of shallow individual classifiers, such as SVMs, Naive Bayes, and clustering, which have been increasingly supplanted by ensemble methods and neural networks. Even though the semi-supervised paradigm accounts for a smaller portion of works, an upward trend in these solutions has been identified, aiming to mitigate ground-truth scarcity. We uphold this tendency by encouraging future research to explore label-agnostic anomaly detection designs further.

Neural networks still are a minority throughout every learning paradigm. This cohort is primarily limited to CNN, LSTM, and autoencoder implementations. Furthermore, research into graph neural networks, large language models, transformers, and attention mechanisms remains scarce. Given the scale and complexity of real-world data, we advocate for a more extensive investigation of neural networks, particularly deep learning, for intrusion detection systems in dynamic scenarios.

C. DRIFT DETECTION

Active concept drift detection was employed in most of the analyzed literature, while fewer works adopted a passive approach. Among active detection research, the most common solution relies on evaluating the intrusion detection system's classification error. This strategy is intrinsically dependent on promptly available and highly accurate labeled data. Practically, this assumption is often decoupled from operational reality, where the immediate availability of ground truth is the exception rather than the rule.

The reliance on rapid labeling introduces significant bottlenecks, as manual annotation is a cost-intensive task that cannot scale with the throughput of high-speed networks. This delay creates a critical gap in real-time detection, as the computational overhead of processing and labeling traffic in production environments often degrades performance. Consequently, to bridge the lab-to-production gap, research must pivot toward approaches that mitigate ground-truth dependency, such as semi-supervised learning, labeling-delay resilient algorithms, or unsupervised, distribution-based detection.

Most detection was achieved by monitoring data distribution changes, thereby remaining label-agnostic. While unsupervised approaches account for the majority of drift detection methods, these algorithms would benefit from a deeper exploration of similarity metrics, such as Entropy, Jensen-Shannon, Kullback-Leibler, Page-Hinkley, and Wasserstein. This research gap is even more pronounced in semi-supervised, hybrid implementations, which remain significantly under-explored.

Concept drift handling follows the No Free Lunch theorem, where a singular method is unable to outperform all others in every scenario [44]. Thus, drift detection algorithms must be chosen under the constraints of the expected CD type. Once virtual drift encompasses the change in the input distribution while maintaining the decision boundary, this category of shift is best identified by distribution-based methods. Conversely, real drift occurs when the decision boundary is changed without altering the input distribution, providing an advantage to classification error-based techniques. Aiming to address both circumstances, we advocate that future research develop hybrid systems.

Moreover, researchers should address the detection delay, which may create a vulnerability window where the model operates under an obsolete decision boundary. For instance, while the DDM may exhibit lag when encountering gradual drift, the EDDM is specifically designed to identify such occurrences more promptly. Minimizing this latency is crucial, as delayed detection can lead to an accumulation of false alarms and prolonged exposure to evolving threats.

D. DRIFT ADAPTATION

To maintain system performance in dynamic environments, drift adaptation techniques must be considered. Most of the analyzed works employ supervised methods that rely entirely

on labeled data and are not necessarily beholden to the underlying intrusion detection system's learning paradigm. Fewer researchers implement strategies to mitigate the real-world delays and gaps in the labeling process, using both unsupervised and semi-supervised approaches.

Besides ground truth dependency, most works fail to address the degradation of previously learned patterns. Only 25 analyzed papers considered the implications of catastrophic forgetting while aiming to mitigate it. Plasticity may be pursued as a design choice, favoring the natural evolution of data over the stability of knowledge. Nevertheless, insights that are currently obsolete may regain validity as recurring patterns emerge in the data. In the field of network security and anomaly detection, patterns should be preserved as their forgotten counterparts may be mistaken for anomalies if they reappear.

Continuous adaptation can be targeted by poisoning and adversarial attacks, which progressively inject anomalous traffic into the system, exploiting its sensitivity to drift or its retraining logic to gradually shift the decision boundary until malicious activities are classified as legitimate. Without triggering alarms, these strategies deface the system's modeling of legitimate behavior, rendering them ineffective at detecting anomalies. Thus, we advocate that researchers explore defensive techniques such as adversarial skepticism and data sanitization.

1) LABEL DEPENDENCE

Works that showed some level of resilience to label scarcity in the adaptation process are almost evenly divided between unsupervised and semi-supervised. Both paradigms are restricted to model-centric and data & representation algorithms, except for the work of Wu et al. [61], which fell under ensemble-based adaptation in the semi-supervised paradigm.

Considering the insights obtained, we suggest that authors focus their research on less label-dependent adaptation solutions. This can be achieved by implementing unsupervised or semi-supervised methods that are resilient to labeling delays. Works should decouple the learning paradigms of intrusion detection and drift-adaptation algorithms to a greater extent. Dynamic anomaly thresholding should also be further explored, especially in label-agnostic incremental learning solutions.

2) CATASTROPHIC FORGETTING

Most works that addressed catastrophic forgetting implemented buffer replay and its variants. Although some of these works aim to reduce storage overhead by saving fewer yet representative samples or by generating synthetic data, this focus remains narrow to a single algorithm. Even though these improvements obviate the need to retain obsolete data, prolonged training times remain a significant concern, since each incorporation of new data must consider samples that represent past knowledge. Other strategies that are independent of reassuring old information warrant further exploration to minimize memory overhead and latency, such as model regularization and parameter freezing. These approaches are particularly critical for ensuring scalability in high-throughput environments where the cost of retraining on large buffers is prohibitive.

E. METRICS

Evaluating a system's efficiency in performing the task it was designed to address is often done using quantitative metrics. Only 7 surveyed works reported drift-specific measurements, mainly variations of accuracy. These metrics often depend not only on traffic type annotation, but also on some signaling that samples represent a drifted version of an already known class. Besides being laborious and time-consuming, these annotations are often obtained manually, making them prone to errors and unavailability. An automated alternative to measuring and annotating drift is clustering, which can display centroids shifting over time. This demonstration is currently limited to studies that employ cluster-based anomaly detection. Thus, we suggest that future research leverage clustering as a means to demonstrate and measure drift, thereby mitigating dependence on labeled data.

Whereas incremental and online learning encompass the adaptation of 19 works, only 4 scope-specific metrics were identified in 2 works, namely Backward Transfer, Forward Transfer, Lifelong Performance, and Update Time. Consequently, future research should explore more comprehensively incremental-specific metrics to provide a deeper understanding of system evolution over time.

VII. CONCLUSION

The dynamicity of network traffic often reflects on behavioral profiling changes, which can render security systems outdated and ineffective. This misalignment between the learned model and evolving traffic patterns creates a vulnerability window that attackers can exploit to remain undetected. During this interval, the system's ability to distinguish between legitimate fluctuations and malicious intent is compromised, fundamentally degrading operational continuity and integrity of network infrastructure.

To provide a walkthrough on the mitigation of these risks, we presented a thorough empirical literature review on concept-drift-resilient anomaly detection and traffic classification systems in the field of computer network security. This survey builds upon previous studies by comprehensively analyzing the components of state-of-the-art dynamic security systems. These elements comprise benchmarking datasets, modeling intrusion detection systems, concept drift detection, concept drift adaptation, and performance evaluation. This holistic perspective helps researchers identify critical gaps in current adaptive paradigms and offers practitioners a technical roadmap to balance detection accuracy with the operational constraints of real-world networks.

Following the proposed methodology, we selected 69 papers to compose the foundation of our review. The five most used datasets are CIC-IDS2017, KDDCup 1999,

NSL-KDD, CSE-CIC-IDS2018, and IoTID20, none of which exhibits drift in the network traffic concept. Four datasets depict dynamic environments, namely the MAWI Working Group archive, DS2OS Traffic Traces, IP Maliciousness, and UGR'16. Most studies implement explicit drift detection using dedicated algorithms, primarily by monitoring error rate increase, leveraging adaptive windowing, ADWIN integrated with EDDM, and DDM. Adaptation to shifts is majorly achieved via triggered retraining, incremental learning, and ensemble voting weights and structure updating. The implementation of intrusion detection systems is mostly hinged on supervised ensembles, especially random forests and their adaptive counterparts. Neural networks are less frequently used and are generally tied to CNN, LSTM, and autoencoder algorithms. Performance is largely assessed by static metrics, such as accuracy, precision, recall, and F1-score, except for 8 drift-specific indicators present in 7 works.

Although the field has seen a rise in scientific production regarding dynamic intrusion detection systems, some challenges remain unaddressed. We have identified critical issues that are still open and warrant further investigation, structured into a research agenda to guide future developments. In the short term, researchers should focus on overcoming the scarcity of drift-oriented datasets, which are essential for the reliable evaluation of dynamic models. Performance measurement could be improved by employing clustering techniques to estimate data shifts, thereby relying on pseudo-labels instead of ground truth. Incremental learning-specific metrics should also be further studied to provide insights into the system's performance under evolution, while unsupervised detection should further include underexplored data distribution metrics, such as Entropy, Jensen-Shannon, Kullback-Leibler, Page-Hinkley, and Wasserstein.

Medium-term challenges involve enhancing operational resilience; even though deep learning should be more widely employed, future frameworks must prioritize resilience to delayed or unavailable labels, addressing the high computational costs and real-time constraints found in real-world environments. This can be accomplished through semi-supervised hybrid approaches, which rely partially on ground truth, and unsupervised methods that remain entirely independent of labels for anomaly and drift detection and adaptation. Furthermore, adaptation algorithms should be more decoupled from the underlying NIDS learning paradigms and explore dynamic anomaly-score thresholding within incremental learning environments. Implementing dynamic NIDS must place greater emphasis on the mitigation of drift detection and adaptation delay, as well as catastrophic forgetting, especially by alternatives beyond buffer replay to ensure scalability.

In the long term, the focus must shift toward adversarial robustness and deployment at scale. Future research should investigate how continuous adaptation logic can be hardened against adversarial behavior and poisoning attacks that exploit drift sensitivity. Finally, balancing these adaptive capabilities with data privacy legislations and the need for low-latency processing in high-speed networks remains the ultimate frontier for making concept-drift-resilient NIDS a staple of modern cybersecurity infrastructure.

REFERENCES

[1] R. Z. Ahmad, M. Rizwan, M. J. Yousuf, M. B. Khan, A. Almadhor, T. R. Gadekallu, and S. Abbas, "Device-to-device communication in 5G heterogeneous network based on game-theoretic approaches: A comprehensive survey," *J. Netw. Comput. Appl.*, vol. 238, Jun. 2025, Art. no. 104152, doi: 10.1016/j.jnca.2025.104152.

[2] M. S. Akbar, Z. Hussain, M. Ikram, Q. Z. Sheng, and S. C. Mukhopadhyay, "On challenges of sixth-generation (6G) wireless networks: A comprehensive survey of requirements, applications, and security issues," *J. Netw. Comput. Appl.*, vol. 233, Jan. 2025, Art. no. 104040, doi: 10.1016/j.jnca.2024.104040.

[3] A. Chebaane, D. Scholze, Y. Rezgui, A. Abdennebi, A. Al-Khatib, A. Khelil, L. Badraoui, and C. Liu, "Fine-grained latency analysis of real-world 5G-enabled multi-tier edge computing for school zone traffic safety," *Discover Internet Things*, vol. 5, no. 1, p. 112, Oct. 2025, doi: 10.1007/s43926-025-00218-1.

[4] S. Singh, E. E. Sham, and D. P. Vidyarthi, "Optimizing fog device deployment for maximal network connectivity and edge coverage using metaheuristic algorithm," *Future Gener. Comput. Syst.*, vol. 157, pp. 529–542, Aug. 2024, doi: 10.1016/j.future.2024.04.010.

[5] M. Le, T. Huynh-The, T. Do-Duy, T.-H. Vu, W.-J. Hwang, and Q.-V. Pham, "Applications of distributed machine learning for the Internet-of-Things: A comprehensive survey," *IEEE Commun. Surveys Tuts.*, vol. 27, no. 2, pp. 1053–1100, Apr. 2025, doi: 10.1109/COMST.2024.3427324.

[6] Sakshi, A. Malik, and A. K. Sharma, "A survey on blockchain based IoT forensic evidence preservation: Research trends and current challenges," *Multimedia Tools Appl.*, vol. 83, no. 14, pp. 42413–42458, Oct. 2023, doi: 10.1007/s11042-023-17104-z.

[7] L. Wang, J. Liu, Q. He, F. Chen, and X. Xia, "Hierarchical runtime reliability anomaly detection for edge services rejuvenation," *IEEE Trans. Mobile Comput.*, vol. 25, no. 4, pp. 5807–5823, Apr. 2026, doi: 10.1109/TMC.2025.3632794.

[8] Q. Yang, W. He, M. Chen, H. Du, S. Shao, F. Wu, S. Liu, Y. Ji, and K. Ren, "End-to-end open-set semi-supervised learning for fine-grained encrypted traffic classification," *IEEE Trans. Inf. Forensics Security*, vol. 21, pp. 1347–1362, 2026, doi: 10.1109/TIFS.2026.3653575.

[9] V. G. da Silva Ruffo, D. M. B. Lent, M. Komarchesqui, V. F. Schiavon, M. V. O. de Assis, L. F. Carvalho, and M. L. Proença, "Anomaly and intrusion detection using deep learning for software-defined networks: A survey," *Expert Syst. Appl.*, vol. 256, Dec. 2024, Art. no. 124982, doi: 10.1016/j.eswa.2024.124982.

[10] M. Lefoane, I. Ghafir, S. Kabir, and I.-U. Awan, "Internet of Things botnets: A survey on artificial intelligence based detection techniques," *J. Netw. Comput. Appl.*, vol. 236, Apr. 2025, Art. no. 104110, doi: 10.1016/j.jnca.2025.104110.

[11] M. Shen, K. Ye, X. Liu, L. Zhu, J. Kang, S. Yu, Q. Li, and K. Xu, "Machine learning-powered encrypted network traffic analysis: A comprehensive survey," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 1, pp. 791–824, 1st Quart., 2023, doi: 10.1109/COMST.2022.3208196.

[12] V. S. D. Priya, S. C. Sethuraman, and M. K. Khan, "Blockchain-based deep learning models for intrusion detection in industrial control systems: Frameworks and open issues," *J. Netw. Comput. Appl.*, vol. 243, Nov. 2025, Art. no. 104286, doi: 10.1016/j.jnca.2025.104286.

[13] M. A. Shyaa, N. F. Ibrahim, Z. B. Zainol, R. Abdullah, M. Anbar, and L. Alzubaidi, "IGPC-MSOS: A knowledge-preserving transfer learning framework with dynamic mode-switching for handling concept drift in network intrusion detection systems," *Knowledge-Based Syst.*, vol. 337, Mar. 2026, Art. no. 115361, doi: 10.1016/j.knosys.2026.115361.

[14] F. Bayram, B. S. Ahmed, and A. Kassler, "From concept drift to model degradation: An overview on performance-aware drift detectors," *Knowledge-Based Syst.*, vol. 245, Jun. 2022, Art. no. 108632, doi: 10.1016/j.knosys.2022.108632.

[15] I. Martins, J. S. Resende, P. R. Sousa, S. Silva, L. Antunes, and J. Gama, "Host-based IDS: A review and open issues of an anomaly detection system in IoT," *Future Gener. Comput. Syst.*, vol. 133, pp. 95–113, Aug. 2022, doi: 10.1016/j.future.2022.03.001.

[16] R. Chu, M. Luo, L. Yang, J. Xiao, and Y. Liao, "Concept drift analysis based on isolation forest for effectively detecting network attack in IoT scenarios," *IEEE Access*, vol. 14, pp. 3245–3258, 2026, doi: 10.1109/ACCESS.2025.3650576.

[17] M. A. Shyaa, N. F. Ibrahim, Z. Zainol, R. Abdullah, M. Anbar, and L. Alzubaidi, "Evolving cybersecurity frontiers: A comprehensive survey on concept drift and feature dynamics aware machine and deep learning in intrusion detection systems," *Eng. Appl. Artif. Intell.*, vol. 137, Nov. 2024, Art. no. 109143, doi: 10.1016/j.engappai.2024.109143.

[18] C. Zha, Z. Wang, Y. Fan, B. Bai, Y. Zhang, S. Shi, and R. Zhang, "A-NIDS: Adaptive network intrusion detection system based on clustering and stacked CTGAN," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 3204–3219, 2025, doi: 10.1109/TIFS.2025.3551643.

[19] A. M. Paim and F. Enembreck, "Adaptive random tree ensemble for evolving data stream classification," *Knowl.-Based Syst.*, vol. 309, Jan. 2025, Art. no. 112830, doi: 10.1016/j.knosys.2024.112830.

[20] X. Gu, G. Howells, and H. Yuan, "A semi-supervised soft prototype-based autonomous fuzzy ensemble system for network intrusion detection," *Appl. Soft Comput.*, vol. 170, Feb. 2025, Art. no. 112719, doi: 10.1016/j.asoc.2025.112719.

[21] F. Cerasuolo, G. Bovenzi, D. Ciuonzo, and A. Pescapè, "Adaptable, incremental, and explainable network intrusion detection systems for Internet of Things," *Eng. Appl. Artif. Intell.*, vol. 144, Mar. 2025, Art. no. 110143, doi: 10.1016/j.engappai.2025.110143.

[22] H. Qiao, B. Novikov, and J. O. Blech, "Concept drift analysis by dynamic residual projection for effectively detecting botnet cyber-attacks in IoT scenarios," *IEEE Trans. Ind. Informat.*, vol. 18, no. 6, pp. 3692–3701, Jun. 2022, doi: 10.1109/TII.2021.3108464.

[23] K. Faber, R. Corizzo, B. Sniezynski, and N. Japkowicz, "VLAD: Task-agnostic VAE-based lifelong anomaly detection," *Neural Netw.*, vol. 165, pp. 248–273, Aug. 2023, doi: 10.1016/j.neunet.2023.05.032.

[24] C. Nixon, M. Sedky, J. Champion, and M. Hassan, "SALAD: A split active learning based unsupervised network data stream anomaly detection method using autoencoders," *Expert Syst. Appl.*, vol. 248, Aug. 2024, Art. no. 123439, doi: 10.1016/j.eswa.2024.123439.

[25] J. Zhang, Z. Chen, H. Cheng, J. Li, and G. Min, "Improving multi-model anomaly traffic detection in MEC networks with large-model- powered continuous learning," *IEEE Netw.*, vol. 39, no. 3, pp. 56–62, May 2025, doi: 10.1109/MNET.2025.3532109.

[26] Z. Niu, J. Xue, Y. Wang, T. Lei, W. Han, and X. Gao, "QARF: A novel malicious traffic detection approach via online active learning for evolving traffic streams," *Chin. J. Electron.*, vol. 33, no. 3, pp. 645–656, May 2024, doi: 10.23919/cje.2022.00.360.

[27] M. Riess, "Automating model management: A survey on metaheuristics for concept-drift adaptation," *J. Data, Inf. Manage.*, vol. 4, nos. 3–4, pp. 211–229, Dec. 2022, doi: 10.1007/s42488-022-00075-5.

[28] S. Agrahari and A. K. Singh, "Concept drift detection in data stream mining: A literature review," *J. King Saud Univ. - Comput. Inf. Sci.*, vol. 34, no. 10, pp. 9523–9540, Nov. 2022, doi: 10.1016/j.jksuci.2021.11.006.

[29] A. L. Suárez-Cetrulo, D. Quintana, and A. Cervantes, "A survey on machine learning for recurring concept drifting data streams," *Expert Syst. Appl.*, vol. 213, Mar. 2023, Art. no. 118934, doi: 10.1016/j.eswa.2022.118934.

[30] O. A. Mahdi, N. Ali, E. Pardede, A. Alazab, T. Al-Quraishi, and B. Das, "Roadmap of concept drift adaptation in data stream mining, years later," *IEEE Access*, vol. 12, pp. 21129–21146, 2024, doi: 10.1109/ACCESS.2024.3358817.

[31] D. Lukats, O. Zielinski, A. Hahn, and F. Stahl, "A benchmark and survey of fully unsupervised concept drift detectors on real-world data streams," *Int. J. Data Sci. Analytics*, vol. 19, no. 1, pp. 1–31, Jan. 2025, doi: 10.1007/s41060-024-00620-y.

[32] P. Moriano, S. C. Hespeler, M. Li, and M. Mahbub, "Adaptive anomaly detection for identifying attacks in cyber-physical systems: A systematic literature review," *Artif. Intell. Rev.*, vol. 58, no. 9, p. 283, Jun. 2025, doi: 10.1007/s10462-025-11292-w.

[33] M. Abdullahi, H. Alhussian, N. Aziz, S. J. Abdulkadir, Y. Baashar, A. A. Alashhab, and A. Afrin, "A systematic literature review of concept drift mitigation in time-series applications," *IEEE Access*, vol. 13, pp. 119380–119410, 2025, doi: 10.1109/ACCESS.2025.3587231.

[34] C. M. Bishop and N. M. Nasrabadi, *Pattern Recognition and Machine Learning*. Cham, Switzerland: Springer, 2006.

[35] I. Goodfellow, Y. Bengio, and A. Courville, "Deep learning," MIT Press, Cambridge, MA, USA, Tech. Rep. 978-0262035613, 2016.

[36] H. Huang, P. Wang, J. Pei, J. Wang, S. Alexanian, and D. Niyato, "Deep learning advancements in anomaly detection: A comprehensive survey," *IEEE Internet Things J.*, vol. 12, no. 21, pp. 44318–44342, Nov. 2025, doi: 10.1109/JIOT.2025.3585884.

[37] L. Xu, Z. Han, D. Zhao, X. Li, F. Yu, and C. Chen, "Addressing concept drift in IoT anomaly detection: Drift detection, interpretation, and adaptation," *IEEE Trans. Sustain. Comput.*, vol. 9, no. 6, pp. 913–924, Nov. 2024, doi: 10.1109/TSUSC.2024.3386667.

[38] N. Malekghaini, E. Akbari, M. A. Salahuddin, N. Limam, R. Boutaba, B. Mathieu, S. Moteau, and S. Tuffin, "Deep learning for encrypted traffic classification in the face of data drift: An empirical study," *Comput. Netw.*, vol. 225, Apr. 2023, Art. no. 109648, doi: 10.1016/j.comnet.2023.109648.

[39] G. Maciá-Fernández, J. Camacho, R. Magán-Carrión, P. García-Teodoro, and R. Therón, "UGR-16: A new dataset for the evaluation of cyclostationarity-based network IDSs," *Comput. Secur.*, vol. 73, pp. 411–424, Mar. 2018, doi: 10.1016/j.cose.2017.11.004.

[40] S. U. Din, Q. Yang, J. Shao, C. B. Mawuli, A. Ullah, and W. Ali, "Synchronization-based semi-supervised data streams classification with label evolution and extreme verification delay," *Inf. Sci.*, vol. 678, Sep. 2024, Art. no. 120933, doi: 10.1016/j.ins.2024.120933.

[41] A. Kuppa and N.-A. Le-Khac, "Learn to adapt: Robust drift detection in security domain," *Comput. Electr. Eng.*, vol. 102, Sep. 2022, Art. no. 108239, doi: 10.1016/j.compeleceng.2022.108239.

[42] A. M. Pasikhani, J. A. Clark, and P. Gope, "Adversarial RL-based IDS for evolving data environment in 6LoWPAN," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 3831–3846, 2022, doi: 10.1109/TIFS.2022.3214099.

[43] A. Bernardo and E. D. Valle, "An extensive study of C-SMOTE, a continuous synthetic minority oversampling technique for evolving data streams," *Expert Syst. Appl.*, vol. 196, Jun. 2022, Art. no. 116630, doi: 10.1016/j.eswa.2022.116630.

[44] H. Hu, M. Kantardzic, and T. S. Sethi, "No free lunch theorem for concept drift detection in streaming data classification: A review," *WIREs Data Mining Knowl. Discovery*, vol. 10, no. 2, p. 1327, Mar. 2020, doi: 10.1002/widm.1327.

[45] R. R. dos Santos, E. K. Viegas, A. O. Santin, and P. Tedeschi, "Federated learning for reliable model updates in network-based intrusion detection," *Comput. Secur.*, vol. 133, Oct. 2023, Art. no. 103413, doi: 10.1016/j.cose.2023.103413.

[46] H. Wu, W. Li, X. Chen, G. Cheng, X. Hu, and Y. Zhuang, "Batch classifier with adaptive update for backbone traffic classification," *Comput. Commun.*, vol. 202, pp. 57–72, Mar. 2023, doi: 10.1016/j.comcom.2023.02.013.

[47] O. A. Wahab, "Intrusion detection in the IoT under data and concept drifts: Online deep learning approach," *IEEE Internet Things J.*, vol. 9, no. 20, pp. 19706–19716, Oct. 2022, doi: 10.1109/JIOT.2022.3167005.

[48] D. E. García and N. DeCastro-García, "Conformal prediction for labelling and updating online models in the presence of concept drift in cybersecurity," *J. Inf. Secur. Appl.*, vol. 93, Sep. 2025, Art. no. 104120, doi: 10.1016/j.jisa.2025.104120.

[49] M. Shajari, H. Geng, K. Hu, and A. Leon-Garcia, "Tensor-based online network anomaly detection and diagnosis," *IEEE Access*, vol. 10, pp. 85792–85817, 2022, doi: 10.1109/ACCESS.2022.3197651.

[50] K. Cho, K. Mitsuya, and A. Kato, "Traffic data repository at the WIDE project [dataset]," in *Proc. USENIX Annu. Tech. Conf. (USENIX ATC)*, Jun. 2000, pp. 263–270. [Online]. Available: https://www.usenix.org/conference/2000-usenix-annual-technical-conference/traffic-data-repository-wide-project.

[51] F. Aubet and M. Pahl. (2018). *DS2OS Traffic Traces [Dataset]*. Accessed: Jan. 19, 2026. [Online]. Available: https://www.kaggle.com/datasets/francoisxa/ds2ostraffictraces.

[52] D. E. García and N. DeCastro-García, "Transfer and online learning for IP maliciousness prediction in a concept drift scenario," *Wireless Netw.*, vol. 30, no. 9, pp. 7423–7444, Dec. 2024, doi: 10.1007/s11276-024-03664-x.

[53] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Inf. Syst. Secur. Privacy*, 2018, pp. 108–116, doi: 10.5220/0006639801080116.

[54] (1999). *KDD Cup 1999 Data [Dataset]*. Accessed: Jan. 19, 2026. [Online]. Available: https://kdd.ics.uci.edu/databases/kddcup99/kddcup99.html.

[55] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in *Proc. IEEE Symp. Comput. Intell. Secur. Defense Appl.*, Jul. 2009, pp. 1–6, doi: 10.1109/CISDA.2009.5356528.

[56] M. Liu, P. Wang, Z. Li, Y. Ye, Z. Wang, and X. Chen, "UDDA-TC: Unsupervised real-time drift detection and adaptation for continual traffic classification in mobile edge computing," *IEEE Trans. Consum. Electron.*, vol. 71, no. 3, pp. 8940–8952, Aug. 2025, doi: 10.1109/TCE.2025.3579882.

[57] G. Liao, P. Zhang, H. Yin, X. Deng, Y. Li, H. Zhou, and D. Zhao, "A novel semi-supervised classification approach for evolving data streams," *Expert Syst. Appl.*, vol. 215, Apr. 2023, Art. no. 119273, doi: 10.1016/j.eswa.2022.119273.

[58] Y. Huang and M. Ma, "AILL-IDS: An automatic incremental lifetime learning intrusion detection system for vehicular ad hoc networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 26, no. 2, pp. 2669–2678, Feb. 2025, doi: 10.1109/TITS.2024.3510584.

[59] A. J. Rabash, M. Z. A. Nazri, A. Shapii, and M. K. Hasan, "Non-dominated sorting genetic algorithm-based dynamic feature selection for intrusion detection system," *IEEE Access*, vol. 11, pp. 125080–125093, 2023, doi: 10.1109/ACCESS.2023.3328395.

[60] N. Samadi, J. Tanha, and M. Jalili, "Graph theory-based semi-supervised self-training for data stream classification and emerging class detection," *Inf. Sci.*, vol. 698, Apr. 2025, Art. no. 121762, doi: 10.1016/j.ins.2024.121762.

[61] Z. Wu, P. Gao, L. Cui, and J. Chen, "An incremental learning method based on dynamic ensemble RVM for intrusion detection," *IEEE Trans. Netw. Service Manage.*, vol. 19, no. 1, pp. 671–685, Mar. 2022, doi: 10.1109/TNSM.2021.3102388.

[62] B. Li, Y. Wang, K. Xu, L. Cheng, and Z. Qin, "DFAID: Density-aware and feature-deviated active intrusion detection over network traffic streams," *Comput. Secur.*, vol. 118, Jul. 2022, Art. no. 102719, doi: 10.1016/j.cose.2022.102719.

[63] Z. Yin, H. Chen, H. Ma, T. Hu, and L. Bai, "CAEAID: An incremental contrast learning-based intrusion detection framework for IoT networks," *Comput. Netw.*, vol. 262, May 2025, Art. no. 111161, doi: 10.1016/j.comnet.2025.111161.

[64] M. Gudapati, A. Borah, and S. R. K., "Incremental approach: A workflow model for semi-supervised learning environment," *Eng. Appl. Artif. Intell.*, vol. 160, Nov. 2025, Art. no. 111815, doi: 10.1016/j.engappai.2025.111815.

[65] M. Saqib, H. Elbiaze, and R. H. Glitho, "A data-driven approach to mitigate evolving volumetric attacks in programmable networks," *IEEE Trans. Mach. Learn. Commun. Netw.*, vol. 3, pp. 883–902, 2025, doi: 10.1109/TMLCN.2025.3594659.

[66] M. Jain, G. Kaur, and V. Saxena, "A K-means clustering and SVM based hybrid concept drift detection technique for network anomaly detection," *Expert Syst. Appl.*, vol. 193, May 2022, Art. no. 116510, doi: 10.1016/j.eswa.2022.116510.

[67] G. F. Scaranti, L. F. Carvalho, S. Barbon, J. Lloret, and M. L. Proença, "Unsupervised online anomaly detection in software defined network environments," *Expert Syst. Appl.*, vol. 191, Apr. 2022, Art. no. 116225, doi: 10.1016/j.eswa.2021.116225.

[68] B. H. Schwengber, A. Vergütz, N. G. Prates, and M. Nogueira, "Learning from network data changes for unsupervised botnet detection," *IEEE Trans. Netw. Service Manage.*, vol. 19, no. 1, pp. 601–613, Mar. 2022, doi: 10.1109/TNSM.2021.3109076.

[69] M. Soltani, K. Khajavi, M. J. Siavoshani, and A. H. Jahangir, "A multi-agent adaptive deep learning framework for online intrusion detection," *Cybersecurity*, vol. 7, no. 1, p. 9, May 2024, doi: 10.1186/s42400-023-00199-0.

[70] T. Qin, G. Cheng, Z. Yin, Y. Wei, Z. Yao, and Z. Chen, "Building robust traffic classifier under low quality data: A federated contrastive learning approach," *Digit. Commun. Netw.*, vol. 11, no. 5, pp. 1479–1492, Oct. 2025, doi: 10.1016/j.dcan.2025.06.001.

[71] M. Eskandari and H. Khotanlou, "Data stream classification using a deep transfer learning method based on extreme learning machine and recurrent neural network," *Multimedia Tools Appl.*, vol. 83, no. 23, pp. 63213–63241, Jan. 2024, doi: 10.1007/s11042-023-18075-x.

[72] S. Cai, H. Tang, J. Chen, Y. Hu, and W. Guo, "CDDA-MD: An efficient malicious traffic detection method based on concept drift detection and adaptation technique," *Comput. Secur.*, vol. 148, Jan. 2025, Art. no. 104121, doi: 10.1016/j.cose.2024.104121.

[73] A. Singh, P. Mishra, P. Vinod, A. Gaur, and M. Conti, "SFC-NIDS: A sustainable and explainable flow filtering based concept drift-driven security approach for network introspection," *Cluster Comput.*, vol. 27, no. 7, pp. 9757–9782, Oct. 2024, doi: 10.1007/s10586-024-04444-0.

[74] A. M. Eldhai, M. Hamdan, A. Abdelaziz, I. A. T. Hashem, S. F. Babiker, M. N. Marsono, M. Hamzah, and N. Z. Jhanjhi, "Improved feature selection and stream traffic classification based on machine learning in software-defined networks," *IEEE Access*, vol. 12, pp. 34141–34159, 2024, doi: 10.1109/ACCESS.2024.3370435.

[75] Z. Niu, J. Xue, D. Qu, Y. Wang, J. Zheng, and H. Zhu, "A novel approach based on adaptive online analysis of encrypted traffic for identifying malware in IIoT," *Inf. Sci.*, vol. 601, pp. 162–174, Jul. 2022, doi: 10.1016/j.ins.2022.04.018.

[76] A. Abdulboriy and J. S. Shin, "An incremental majority voting approach for intrusion detection system based on machine learning," *IEEE Access*, vol. 12, pp. 18972–18986, 2024, doi: 10.1109/ACCESS.2024.3361041.

[77] Y. Wu, L. Liu, Y. Yu, G. Chen, and J. Hu, "Online ensemble learning-based anomaly detection for IoT systems," *Appl. Soft Comput.*, vol. 173, Apr. 2025, Art. no. 112931, doi: 10.1016/j.asoc.2025.112931.

[78] Y. K. Beshah, S. L. Abebe, and H. M. Melaku, "Multi-stage adversarial defense for online DDoS attack detection system in IoT," *IEEE Access*, vol. 13, pp. 72657–72673, 2025, doi: 10.1109/ACCESS.2025.3560186.

[79] J. Zhang, B. Gong, Q. Wang, Y. Wu, and G. Zheng, "An intelligent edge dual-structure ensemble method for data stream detection and releasing," *IEEE Internet Things J.*, vol. 11, no. 1, pp. 863–879, Jan. 2024, doi: 10.1109/JIOT.2023.3286185.

[80] S. Saravanan and U. M. Balasubramanian, "An adaptive scalable data pipeline for multiclass attack classification in large-scale IoT networks," *Big Data Mining Analytics*, vol. 7, no. 2, pp. 500–511, Jun. 2024, doi: 10.26599/bdma.2023.9020027.

[81] L. Yang, S. Naser, A. Shami, S. Muhaidat, L. Ong, and M. Debbah, "Toward zero touch networks: Cross-layer automated security solutions for 6G wireless networks," *IEEE Trans. Commun.*, vol. 73, no. 9, pp. 7650–7679, Sep. 2025, doi: 10.1109/TCOMM.2025.3547764.

[82] W. Liu, C. Zhu, Z. Ding, H. Zhang, and Q. Liu, "Multiclass imbalanced and concept drift network traffic classification framework based on online active learning," *Eng. Appl. Artif. Intell.*, vol. 117, Jan. 2023, Art. no. 105607, doi: 10.1016/j.engappai.2022.105607.

[83] N. Malathy, A. T. Swvtha, T. B. Leela, and A. Raaman, "Real-time intrusion detection in IIoT stream data using window-based weighted ensemble techniques," *Social Netw. Comput. Sci.*, vol. 6, no. 1, p. 66, Jan. 2025, doi: 10.1007/s42979-024-03597-4.

[84] J. Jiang, F. Liu, Y. Liu, Q. Tang, B. Wang, G. Zhong, and W. Wang, "A dynamic ensemble algorithm for anomaly detection in IoT imbalanced data streams," *Comput. Commun.*, vol. 194, pp. 250–257, Oct. 2022, doi: 10.1016/j.comcom.2022.07.034.

[85] A. A. Alashhab, M. S. Zahid, B. Isyaku, A. A. Elnour, W. Nagmeldin, A. Abdelmaboud, T. A. A. Abdullah, and U. D. Maiwada, "Enhancing DDoS attack detection and mitigation in SDN using an ensemble online machine learning model," *IEEE Access*, vol. 12, pp. 51630–51649, 2024, doi: 10.1109/ACCESS.2024.3384398.

[86] M. A. Shyaa, N. F. Ibrahim, Z. B. Zainol, R. Abdullah, and M. Anbar, "Reinforcement learning-based voting for feature drift-aware intrusion detection: An incremental learning framework," *IEEE Access*, vol. 13, pp. 37872–37903, 2025, doi: 10.1109/ACCESS.2025.3544221.

[87] F.-Q. Li, R.-J. Zhao, S.-L. Wang, L.-B. Chen, A. W.-C. Liew, and W. Ding, "Online intrusion detection for Internet of Things systems with full Bayesian possibilistic clustering and ensembled fuzzy classifiers," *IEEE Trans. Fuzzy Syst.*, vol. 30, no. 11, pp. 4605–4617, Nov. 2022, doi: 10.1109/TFUZZ.2022.3165390.

[88] T. Anithakumari and S. Mishra, "OASIS: Online adaptive ensembles for drift adaptation on evolving IoT data streams," *Internet Things*, vol. 31, May 2025, Art. no. 101545, doi: 10.1016/j.iot.2025.101545.

[89] M. Amin, F. Al-Obeidat, A. Tubaishat, B. Shah, S. Anwar, and T. A. Tanveer, "Cyber security and beyond: Detecting malware and concept drift in AI-based sensor data streams using statistical techniques," *Comput. Electr. Eng.*, vol. 108, May 2023, Art. no. 108702, doi: 10.1016/j.compeleceng.2023.108702.

[90] M. S. Noori, R. K. Z. Sahbudin, A. Sali, and F. Hashim, "Feature drift aware for intrusion detection system using developed variable length particle swarm optimization in data stream," *IEEE Access*, vol. 11, pp. 128596–128617, 2023, doi: 10.1109/ACCESS.2023.3333000.

[91] S. Suryawanshi, A. Goswami, and P. Patil, "CDA-PDDWE: Concept drift-aware performance-based diversified dynamic weighted ensemble for non-stationary environments," *Arabian J. Sci. Eng.*, vol. 49, no. 9, pp. 12989–13004, Sep. 2024, doi: 10.1007/s13369-024-08929-3.

[92] M. Abbasi, S. L. Flórez, A. Shahraki, A. Taherkordi, J. Prieto, and J. M. Corchado, "Class imbalance in network traffic classification: An adaptive weight ensemble-of-ensemble learning method," *IEEE Access*, vol. 13, pp. 26171–26192, 2025, doi: 10.1109/ACCESS.2025.3538170.

[93] R. Wang, H. Qiu, X. Cheng, and X. Liu, "Anomaly detection with a container-based stream processing framework for industrial Internet of Things," *J. Ind. Inf. Integr.*, vol. 35, Oct. 2023, Art. no. 100507, doi: 10.1016/j.jii.2023.100507.

[94] J. Cao, X. Di, X. Liu, J. Li, Z. Li, L. Zhao, A. Hawbani, and M. Guizani, "Anomaly detection for in-vehicle network using self-supervised learning with vehicle-cloud collaboration update," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 7, pp. 7454–7466, Jul. 2024, doi: 10.1109/TITS.2024.3351438.

[95] M. Raeiszadeh, A. Ebrahimzadeh, R. H. Glitho, J. Eker, and R. A. F. Mini, "Real-time adaptive anomaly detection in industrial IoT environments," *IEEE Trans. Netw. Service Manage.*, vol. 21, no. 6, pp. 6839–6856, Dec. 2024, doi: 10.1109/TNSM.2024.3447532.

[96] Z. Zhao, Z. Li, X. Xie, J. Yu, F. Zhang, R. Zhang, B. Chen, X. Luo, M. Hu, and W. Ma, "FOSS: Towards fine-grained unknown class detection against the open-set attack spectrum with variable legitimate traffic," *IEEE/ACM Trans. Netw.*, vol. 32, no. 5, pp. 3945–3960, Oct. 2024, doi: 10.1109/TNET.2024.3413789.

[97] A. M. Pasikhan, J. A. Clark, and P. Gope, "Incremental hybrid intrusion detection for 6LoWPAN," *Comput. Secur.*, vol. 135, Dec. 2023, Art. no. 103447, doi: 10.1016/j.cose.2023.103447.

[98] A. Nitish, J. Hanumanthappa, S. P. S. Prakash, and K. Krinkin, "Class imbalance and concept drift invariant online botnet threat detection framework for heterogeneous IoT edge," *Comput. Secur.*, vol. 141, Jun. 2024, Art. no. 103820, doi: 10.1016/j.cose.2024.103820.

[99] Q. Lu, K. An, J. Li, and J. Wang, "Network intrusion detection for modern smart grids based on adaptive online incremental learning," *IEEE Trans. Smart Grid*, vol. 16, no. 3, pp. 2541–2553, May 2025, doi: 10.1109/TSG.2025.3535949.

[100] S. Selvam and U. M. Balasubramanian, "UASDAC: An unsupervised adaptive scalable DDoS attack classification in large-scale IoT network under concept drift," *IEEE Access*, vol. 12, pp. 64701–64716, 2024, doi: 10.1109/ACCESS.2024.3397512.

[101] L. Tang, R. Wei, B. Xia, Y. Tang, W. Wang, Q. Huang, and Q. Chen, "Online anomaly detection in industrial IoT networks using a supervised contrastive learning-based spatiotemporal variational autoencoder," *IEEE Internet Things J.*, vol. 12, no. 11, pp. 17399–17412, Jun. 2025, doi: 10.1109/JIOT.2025.3537864.

[102] X. Lin, G. Xiong, G. Gou, W. Dong, J. Yu, Z. Li, and W. Xia, "Respond to change with constancy: Instruction-tuning with LLM for non-I.I.D. Network traffic classification," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 5758–5773, 2025, doi: 10.1109/TIFS.2025.3574971.

[103] P. S. Mrudula, R. D. A. Raj, A. Pallakonda, Y. R. M. Reddy, K. K. Prakasha, and V. Anandkumar, "Smart grid intrusion detection for IEC 60870-5-104 with feature optimization, privacy protection, and honeypot-firewall integration," *IEEE Access*, vol. 13, pp. 128938–128958, 2025, doi: 10.1109/ACCESS.2025.3590151.

[104] S. A. Elsaid and A. Binbusayyis, "An optimized isolation forest based intrusion detection system for heterogeneous and streaming data in the industrial Internet of Things (IIoT) networks," *Discover Appl. Sci.*, vol. 6, no. 9, p. 483, Sep. 2024, doi: 10.1007/s42452-024-06165-w.

[105] D. J. Kalita, V. P. Singh, and V. Kumar, "A novel adaptive optimization framework for SVM hyper-parameters tuning in non-stationary environment: A case study on intrusion detection system," *Expert Syst. Appl.*, vol. 213, Mar. 2023, Art. no. 119189, doi: 10.1016/j.eswa.2022.119189.

[106] L. Wang, S. Chen, X. Zhang, and J. Liu, "Runtime reliability fractional distribution change analytics against cloud-based systems DDoS attacks," *J. Syst. Softw.*, vol. 220, Feb. 2025, Art. no. 112265, doi: 10.1016/j.jss.2024.112265.

[107] S. Das, A. M. Pasikhani, P. Gope, J. A. Clark, C. Patel, and B. Sikdar, "AIDPS: Adaptive intrusion detection and prevention system for underwater acoustic sensor networks," *IEEE/ACM Trans. Netw.*, vol. 32, no. 2, pp. 1080–1095, Dec. 2023, doi: 10.1109/TNET.2023.3313156.

[108] M. Baena-Garcıa, J. del Campo-Ávila, R. Fidalgo, A. Bifet, R. Gavalda, and R. Morales-Bueno, "Early drift detection method," in *Proc. 4th Int. Workshop Knowl. Discovery Data Streams*, vol. 6, Berlin, Germany, 2006, pp. 77–86.

[109] S. Grossberg, *How Does a Brain Build a Cognitive Code?*. Dordrecht, The Netherlands: Springer, 1982, pp. 1–52, doi: 10.1007/978-94-009-7758-7_1.



MATEUS KOMARCHESQUI received the B.Sc. degree in computer science from the State University of Londrina (UEL), Brazil, in 2024, where he is currently pursuing the M.Sc. degree in computer science and the Ph.D. degree in electrical engineering. He has been a member of the ORION Research Group, since 2023, where he developed research on intrusion detection using quantum computing, deep learning, and explainable artificial intelligence. His research interests include software-defined networking, network management and security, anomaly detection, concept drift, deep learning, and explainable artificial intelligence.



VITOR GABRIEL DA SILVA RUFFO received the B.Sc. (summa cum laude) and M.Sc. degrees in computer science from the State University of Londrina (UEL), Londrina, Brazil, in 2023 and 2025, respectively, where he is currently pursuing the Ph.D. degree in electrical engineering. He is a Visiting Researcher with the Polytechnic University of Valencia, Gandia, Spain. He has been an Active Member of the ORION Research Group, Computer Science Department, UEL, since 2021. His research interests include network anomaly detection, deep learning, and concept drift.



DANIEL MATHEUS BRANDÃO LENT received the bachelor's and master's degrees from the State University of Londrina (UEL), in 2021 and 2023, respectively. He is currently pursuing the Ph.D. degree in electrical engineering (with a focus on computer network security and deep learning). He is a member of the ORION Research Group, which studies computer networks and data communication with the Computer Science Department, UEL. His research interests include deep learning and network anomaly detection.



VINÍCIUS FERREIRA SCHIAVON received the B.Sc. degree in computer science from the State University of Londrina (UEL), in 2025, where he is currently pursuing the M.Sc. degree in computer science. He has been an Active Member of the Orion Research Group, Department of Computing, UEL, since 2023. His research interests include deep learning, datasets, anomaly detection, and network security.



MATHEUS RIKI NAKANISHI received the B.Sc. degree in computer science from the State University of Londrina (UEL), Brazil, in 2026, where he is currently pursuing the M.Sc. degree. He is a member of the ORION Research Group, which focuses on computer networks and data communications, where he conducted research on anomaly detection in software-defined networks under the influence of concept drift. His research interests include anomaly detection, network management and security, machine learning, and concept drift.



GUSTAVO HIDEYUKI KITAMURA NISHIKAWA received the B.Sc. degree in computer science from the State University of Londrina (UEL), Brazil, in 2026, where he is currently pursuing the M.Sc. degree in computer science. He is a member of the ORION Research Group. His research interests include management and security of computer networks and deep learning.



LUIZ FERNANDO CARVALHO received the master's degree in computer science from the State University of Londrina (UEL), in 2014, and the Ph.D. degree in electrical engineering and telecommunications from the State University of Campinas (UNICAMP), in 2018. He has experience in computer science, with an emphasis on computer networks. He is part of the ORION Research Group and a Professor with the Federal University of Technology—Paraná (UTFPR). His main research interests include management and security of computer networks and software-defined networks.



MARIO LEMES PROENÇA JR. received the M.Sc. degree in computer science from the Informatics Institute, Federal University of Rio Grande do Sul (UFRGS), in 1998, and the Ph.D. degree in electrical engineering and telecommunications from the State University of Campinas (UNICAMP), in 2005. He is an Associate Professor and a Leader of the ORION Research Group, which studies computer networks with the Computer Science Department, State University of Londrina (UEL), Brazil. He has supervised more than 160 Ph.D., Master of Science, graduate, and undergraduate students in computer science. He is a Master's Supervisor in computer science with the Computer Science Department, UEL. Also, he is the Ph.D. Supervisor with the Department of Electrical Engineering, UEL. He has authored or co-authored over 120 papers in refereed international journals and conferences, book chapters, and one software register patent. His research interests include computer networks, network operations, management and security, and IT governance.

Coordenação de Aperfeiçoamento de Pessoal de Nível Superior (CAPES) - ROR identifier: 00x0ma614